מערכת הקלט/פלט (פרק 2)

מערכת הקלט/פלט (I/O) מקשרת בין התוכניות לבין העולם החיצון. מטרותיה העיקריות הן זמן תגובה מהיר, קצב העברה גבוה (Throughput), ניצול משאבים מינימלי, אמינות ונוחות שימוש.

Bus: על הבס יש סדרה של סיביות שמתארת את הכתובת, את הדאטה, האם כותבים או קוראים, וסיביות שמתארות פסיקות.

מבנה החומרה ומנגנוני תקשורת: ההתקנים (דיסק, מקלדת, מסך) מחוברים ל**בקרים** (Controllers) שיושבים על ערוץ התקשורת (Bus). התקשורת מתבצעת בשתי שיטות עיקריות:

1. דגימה (Polling): מערכת ההפעלה דוגמת את הבקר מדי פעם ("האם הגיע מידע?"). יש נטייה להשתמש ברשת, לדוגמא מנגנוני התראות ברשת. אבל כמעט ולא בשימוש.
- **חסרונות:** מבזבז משאבי מעבד (Busy Waiting) או מגיב לאט מדי אם קצב הדגימה נמוך. כמעט ואינו בשימוש במערכות מודרניות.

2. פסיקות (Interrupts): הבקר מודיע למעבד כשיש אירוע (למשל, מקש נלחץ או דיסק סיים לקרוא).
- המעבד משעה את התוכנית הרצה, שומר את מצבה באוגרים ייעודיים, ומריץ **שגרת פסיקה** (ISR) של מערכת ההפעלה. בסוף הוא מחזיר את המצב למה שהיה לפני שגרת הפסיקה.
- **בקר פסיקות:** רכיב חומרה המרכז את הפסיקות מההתקנים השונים, מתעדף אותן ומעביר למעבד (או לליבה ספציפית במעבד מרובה ליבות). במעבדים עם כניסת פסיקה אחת (הרבה מעבדי אינטל) משתמשים בבקר פסיקות שיש לו כמה כניסות פסיקה ויציאה אחת. בקר הפסיקות מודיע למעבד שאחד הבקרים הפעיל פסיקה ומערכת ההפעלה יכולה לברר בעזרתו איזה בקר הפעיל את הפסיקה.
- **מניעת הפעלה רקורסיבית של שגרות פסיקה:** פסיקה משעה את השגרה שרצה ומפעילה את שגרת הפסיקה. לא משעים שגרת פסיקה אם מגיעה שגרה נוספת. מערכות הפעלה מתקדמות יודעות לתעדף פסיקות.
- **הפעלה דחויה (Deferred Procedure):** כדי לא לתקוע את המערכת, שגרת הפסיקה מבצעת רק את המינימום ההכרחי (DPC), ושאר הטיפול נדחה למועד מאוחר יותר לפני החזרה לתוכנית המשתמש.

גישה ישירה לזיכרון (DMA): במקום שהמעבד יעתיק נתונים בית-אחרי-בית מהדיסק לזיכרון, בקר ה-DMA מבצע את ההעברה באופן עצמאי. המעבד רק נותן פקודה ("תעביר X בלוקים לכתובת Y") ומקבל פסיקה כשזה הסתיים. זה חוסך עומס עצום על המעבד. בקר ה-DMA יכול להיות יחיד ומחובר לכלל הבקרים האחרים, אבל יכול להיות גם בקר DMA בכל בקר אחר.

מנהלי התקן (Device Drivers): מנהל התקן הוא מודול תוכנה המכיר את הפרטים הטכניים של חומרה ספציפית ומספק למערכת ההפעלה ממשק סטנדרטי (פונקציות כמו `open`, `read`, `write`, `close`, `ioctl`).
- **קבצים מיוחדים (Device Files):** בלינוקס/יוניקס, התקנים מיוצגים כקבצים בתיקייה `/dev/`. תוכניות יכולות לכתוב להתקן כאילו הוא קובץ.
- דוגמה לפקודה: `cat iso > /dev/cdrom`.
- **מספרים מזהים:** Major Number (סוג הדרייבר) זה המספר שמייצג בטבלה של מנהלי ההתקן את מנהל ההתקן הספציפי, ו-Minor Number (המופע הספציפי) המספר הזה הוא ההתקן הספציפי.

דוגמה: סיפורה של גישה לדיסק
- תוכנית מבצעת קריאת מערכת – קריאה מקובץ (שכבר נפתח).
- מערכת ההפעלה מפעילה את שגרת `read` של מנהל ההתקן.
- מנהל ההתקן שולח הוראה בערוץ פלט אל ההתקן: "קרא N בלוקים ממקום M וכתוב את תוכנם לזיכרון בכתובת A" בעזרת בקר DMA.
- ... המתנה (אולי פעילה) ...
- ההתקן קורא מידע מהמדיה ומעביר אותו בגישה ישירה לזיכרון.
- ההתקן יוצר פסיקת חומרה.
- המעבד קופץ לשגרת הפסיקה של מערכת ההפעלה.
- מערכת ההפעלה מריצה את שגרת הפסיקה של מנהל ההתקן:
- שגרה דחויה של מנהל ההתקן מעתיקה את החלק הרלוונטי של המידע בכתובת A אל הזיכרון של התוכנית.
- מנהל ההתקן מדווח למערכת ההפעלה על סיום פעולת הקריאה.
- מערכת ההפעלה חוזרת מקריאת המערכת והתוכנית ממשיכה לרוץ.

שיפור ביצועים: הטמנה וחציצה
- **הטמנה (Caching):** שמירת נתונים שנקראו מהדיסק בזיכרון הראשי (RAM) כדי שקריאה חוזרת תהיה מהירה.
- **חציצה (Buffering):** אגירת נתונים בזיכרון לפני כתיבתם להתקן (למשל, כדי לכתוב לדיסק כשהוא פנוי או במנות גדולות ויעילות).

מנהלי התקן עם וללא הטמנה: block devices - מנהל התקן עם הטמנה.
character devices - מנהל התקן ללא הטמנה.
יש טבלה מסודרת שמייצגת את ה-BLOCK וה-CHARACTER.

אחסון: דיסקים קשיחים (HDD) מול SSD

דיסקים קשיחים (HDD): בנויים מפלטות מגנטיות מסתובבות וזרוע קוראת. צוואר הבקבוק הוא הזזת הזרוע ("זמן חיפוש").

מדיניות תזמון (Scheduling) לדיסק:
- **FCFS:** לפי סדר הגעה. הוגן אך איטי (הזרוע קופצת מצד לצד).
- **SSTF:** משרת את הבקשה הקרובה ביותר למיקום הנוכחי. מהיר מאוד אך עלול לגרום ל**הרעבה** של בקשות רחוקות.
- **Elevator (SCAN):** הזרוע נעה הלוך-ושוב מקצה לקצה ומשרתת בקשות בדרך (כמו מעלית). שילוב טוב של יעילות והוגנות.

וריאצים על מדיניות מעלית:
- **scan:** הזרוע נעה פנימה והחוצה בין שתי המסילות הקיצוניות ומשרתת בקשות שהיא פוגשת בדרך
- **c-scan:** כנ"ל, אבל משרתים בקשות רק בדרך פנימה, את הדרך החוצה עושים במהירות; יותר הוגן ואולי מעט פחות יעיל (מעט כי הזרוע מאיצה)
- **look:** כמו scan אבל הזרוע לא נעה מעבר למסילה הקיצונית שיש עבורה בקשה. יותר יעיל ופחות הוגן
- **c-look:** מובן מאליו
- אפשר להפעיל את מדיניות דומה גם לגבי קטעים בתוך מסילה
- האם כדאי לשרת בקשות למסילה שנמצאים בה שנכנסו לתור לאחר שהזרוע הגיע למסילה. מדוע? שקלו הגינות, יעילות

כונני SSD (Flash): מבוססים על זיכרון ללא חלקים נעים. מהירים הרבה יותר מדיסקים מכניים.
- הזיכרון מחולק לבלוקים ודפים. כתיבה דורשת מחיקה של בלוק שלם.
- **שחיקה:** תאי הזיכרון נהרסים לאחר מספר מסוים של כתיבות.
- **בקר חכם:** מבצע "איזון שחיקה" (Wear Leveling) ומפזר את הכתיבות על פני הכונן, ומחזיק טבלת מיפוי לוגית-פיזית בזיכרון ה-RAM שלו.

SSDs צריכים בקר חכם:
- לא כדאי **לשכתב סקטור במקום**, כי צריך יהיה למחוק בלוק שלם.
- תאי זיכרון **נשחקים** (נהרסים) במהלך כתיבות/מחיקות; מספר מחזורי הכתיבה הוא בין אלף ל-100 אלף (הולך ונהיה גרוע).
- הבקר כותב את סקטור i לדף j וזוכר את **המיפוי**.
- מבנה הנתונים של המיפוי נשמר ב-RAM של הבקר ועל ה-flash.
- הנתונים אודות הנתונים (Metadata) מכילים את זהות הסקטורים השמורים בדף, **קוד לתיקון שגיאות** (ECC), ומידע אחר שהבקר צריך.

RAID (מערך דיסקים): שימוש במספר דיסקים יחד כדי לשפר אמינות ו/או ביצועים:
- **RAID 0 (Striping):** פיזור המידע על פני הדיסקים. ביצועים מעולים, אך אם דיסק אחד מת - כל המידע אבוד (אין יתירות).
- **RAID 1 (Mirroring):** כל מידע נכתב לשני דיסקים במקביל. אמינות גבוהה, אך יקר (מנצל רק 50% מהנפח).
- **RAID 5:** שומר מידע ובלוקי "זוגיות" (Parity) מבוזרים. מאפשר התאוששות מקריסה של דיסק בודד תוך ניצול יעיל של הנפח, אך הכתיבה איטית יותר (בגלל חישוב הזוגיות).

ניהול זיכרון (פרק 3)

מטרות ניהול הזיכרון: מערכת ההפעלה מנהלת את הזיכרון כדי להשיג מספר מטרות קריטיות:
- **הגנה ובידוד:** יכולת להריץ מספר תוכניות בו-זמנית כך שכל תוכנית מוגנת מפני גישה לא מורשית של תוכניות אחרות לזיכרון שלה.
- **זיכרון וירטואלי (מעבר למגבלות פיזיות):** יכולת להריץ בו-זמנית תוכניות שסך הזיכרון שהוקצה להן גדול מהזיכרון הפיזי המותקן במחשב.
- **שימוש בדיסק כהרחבה:** יכולת להשתמש בדיסקים כהרחבה זולה אך איטית של הזיכרון (המעבד אינו יכול להתייחס ישירות למידע בדיסק).
- **הזזה (Relocation):** יכולת להזיז את מבני הנתונים של תוכנית במהלך ריצתה בלי שהתוכנית מודעת לכך. הזזות כאלו מאפשרות לנצל "חורים" קטנים בזיכרון, לאחד חורים לזיכרון פנוי רצוף, או להעביר מבנה נתונים מהזיכרון לדיסק ומשם למקום אחר בזיכרון.

מנגנון הזיכרון הוירטואלי - הפרדת כתובות: קיים ניתוק בין הכתובות שהתוכנית רואה לבין הכתובות בחומרה:
- **כתובת בתוכנית (וירטואלית):** התוכנית מציגה למעבד כתובות של תאי זיכרון (לקריאה/כתיבה או פקודות לביצוע). הכתובות מאוחסנות באוגרים (registers) או בזיכרון. דוגמה לפקודה: `load $r4=*r5`.
- **כתובת על הפס (פיזית):** המעבד אינו מציג את הכתובות הוירטואליות על ערוץ הכתובות בפס, אלא מתרגם אותן לכתובות אחרות תוך שימוש במבנה נתונים שמערכת ההפעלה מקימה ומתחזקת.

תרגום כתובות (Address Translation): המעבד מתרגם כל כתובת וירטואלית לכתובת פיזית:
- **טבלת דפים (Page Table):** מערכת ההפעלה מקימה מבנה נתונים שממפה את הדפים של תהליך למסגרות פיזיות. הטבלה שמורה בזיכרון, ומערכת ההפעלה מודיעה למעבד היכן היא בעזרת אוגר מיוחד, לאחר מכן המעבד קורא מהטבלת דפים.

מי מחליט על המבנה נתונים? היצרן של המעבד שכן הוא זה שצריך לדעת לקרוא אותו
- **TLB (Translation Lookaside Buffer):** על מנת לחסוך את הצורך בחיפוש בטבלה בכל גישה לזיכרון, המעבד שומר בהתקן חומרה מיוחד מיפויים שנעשה בהם שימוש לאחרונה.

אופן התרגום: בהינתן כתובת וירטואלית, המעבד מפרק אותה לשני חלקים:
1. **מספר הדף:** הסיביות הבכירות.
2. **היסט (Offset):** הסיביות הזוטרות (בתוך הדף).

המעבד ממפה את הדף למסגרת פיזית ומשרשר את ההיסט למספר המסגרת ליצירת הכתובת הפיזית.

מבני נתונים לטבלאות דפים: ישנן מספר שיטות לניהול הטבלאות:
1. **טבלה שטוחה (Flat Page Table):** מערך בגודל מספר הדפים במרחב הזיכרון הוירטואלי. כל איבר במערך מציין את מספר המסגרת וסיביות מצב (כמו invalid אם הדף לא מוקצה, או not present אם המסגרת אינה בזיכרון).
   - חיסרון: הטבלה גדולה מאוד. לדוגמה, במרחב של 4GB (מצביעי 32 סיביות) עם דפים של 8KB, נדרשת טבלה עם 512K איברים. אם כל איבר הוא 4 בתים, הטבלה שוקלת 2MB לכל תהליך. במעבדי 64 סיביות שיטה זו אינה ישימה.
2. **טבלה הירכית דלילה (Hierarchical/Sparse):** המיפויים שמורים בעץ חיפוש. מספר הדף מפורק לקבוצות סיביות, כאשר כל קבוצה משמשת אינדקס לרמה אחרת בעץ (רמה 0, רמה 1, וכו'). המיפויים עצמם (העלים) מיוצגים על ידי מערכים.
   - יתרון: חיסכון עצום בזיכרון עבור מרחבים שרק חלק קטן מהם מוקצה (תתי-עצים ללא מיפויים תקפים אינם מיוצגים כלל - המצביע אליהם הוא null).
   - בהילוך על העץ חיפוש כל הילוך זה גישה לזיכרון לכן עץ עמוק יהיה זה חסרון
   - אם מגדילים את הזכרון שיש בכל צומת עצמה, יוצרים מצב שבו יכול ליהיות הרבה זכרון מבוזבז.
3. **טבלה הפוכה (Inverted Page Table):** טבלה אחת לכל התהליכים במערכת. שומרת רק מיפויים של דפים שנמצאים כרגע בזיכרון RAM.
   - ממומשת כטבלת גיבוב (hash) שהמפתח שלה הוא (מספר דף + מספר תהליך לפעמים).
   - עיקר שימושה עבור PAGE_OUT כדי לבצע INVALID ב PT
   - יתרון: גודל הטבלה תלוי רק בגודל הזיכרון הפיזי.
   - חברון: אלגוריתם יחסית מסובך

התייחסות לכתובות פיזיות: מערכת ההפעלה צריכה להתייחס לכתובות פיזיות (עבור טבלאות דפים, בקרים, ו-DMA). דרך פשוטה היא למפות את כל הזיכרון הפיזי (או חלקו) לזיכרון הוירטואלי של כל התהליכים. לדוגמה: כתובת פיזית X ממופה לכתובת וירטואלית $X+2^{31}$. שימוש ב"דפים גדולים" (למשל 4MB) מייעל מיפוי זה.

הרבה מערכות הפעלה בכל טבלת דפים של תהליך היא שומרת מיפוי של של חלק מהזכרון הפיזי שהוא שמור למקרה שצריך לגשת באופן ישיר לזכרון פיזי מיידי בקר או משהו בסגנון

תחזוקת הטבלאות וה-TLB: מערכת ההפעלה מתחזקת את טבלאות הדפים. כאשר משנים כניסה בטבלה, חובה למחוק את המיפוי הישן מה-TLB (בעזרת פקודת מכונה מיוחדת). בעת החלפת תהליך (Context Switch), מערכת ההפעלה מעדכנת את האוגר המצביע לטבלת הדפים.

הגנה ובקרת גישה: אנחנו רוצים שמבנה הנתונים של טבלאות הדפים יהיה מוגן כדי שתהליך אחד לא יוכל לשנות אץ טבלת הדפים של תהליך אחר

מנגנון המיפוי מאפשר לממש בקרת גישה:
- **זיהוי תהליך:** בטבלה הפוכה, המעבד משתמש במזהה התהליך שרץ כדי לאפשר גישה רק למיפויים שלו.
- **סיביות גישה:** כל כניסה בטבלה כוללת הרשאות:
  - האם מותרת גישה במצב משתמש (User) או רק במצב מיוחס (Privileged)?
  - האם מותרת קריאה, כתיבה, וריצה (Execute)?

שימושים: מיפוי מערכת ההפעלה לכל התהליכים אך עם הרשאת "מצב מיוחס בלבד" (ממזער תקורה בקריאות מערכת); גילוי שגיאות על ידי אי-מיפוי של הדפים הראשונים (כתובת 0 / null).

היכולת לאסור גישה במצב משתמש:
- מאפשרת למערכת ההפעלה למפות את הקוד ומבני הנתונים שלה לבי לאפשר לתוכנית שהתהליך מריץ לגשת לזיכרון הזה.
- מערכת ההפעלה ממופה בד"כ לכל התהליכים באותו המקום למשל בGB הראשון העליון של כל מרחב זיכרון ווירטואלי.
- ממזער תקורה בעת קריאות מערכת ופסיקות

חריגי דף (Page Faults): המעבד מייצר חריג (Exception) כאשר יש גישה לדף לא ממופה, דף שבדיסק, או הפרת הרשאות. **הטיפול על ידי מערכת ההפעלה:**
1. **דף חוקי אך בדיסק:** המערכת משעה את התהליך, מביאה את המסגרת מהדיסק, מעדכנת את הטבלה וחוזרת לביצוע הפקודה מחדש.
2. **שגיאת תוכנית:** גישה לדף שאינו מוקצה כלל לתהליך. התוצאה היא סיום התהליך או שליחת איתות (signal).
3. **דף חדש:** הקצאת מסגרת מאופסת.

ניהול מסגרות ופינוי (Eviction): לכל מסגרת פיזית יש מצב:
- **פנויה ריקה (מאופסת):** מוכנה להקצאה מיידית.
- **פנויה נקייה:** מכילה מידע ישן אך לא רלוונטי (לא אופסה).
- **מוקצת נקייה:** זהה לעותק בדיסק.
- **מוקצת מלוכלכת:** שונתה בזיכרון, העותק בדיסק לא מעודכן.

מטרת העל: מאגר תמידי של מסגרות פנויות **מנגנון הפינוי:** תהליכי רקע מתעוררים מדי פעם ומבצעים: העברת מסגרות נקיות לרשימת המועמדות לפינוי; כתיבת מסגרות מלוכלכות לדיסק (והפיכתן לנקיות); ואיפוס מסגרות נקיות למאגר הפנויות. אם תהליך צריך דף מרשימת הפינוי לפני שנדרס, ניתן לבצע "הצלה" (Rescue) ללא גישה לדיסק.

מדיניות החלפת דפים (Replacement Policy): כיצד בוחרים איזה דף לפנות ("קורבן")?
- **מיטבי (Optimal):** לפנות את הדף שהשימוש הבא בו הוא הרחוק ביותר (תיאורטי).
- **LRU (Least Recently Used):** לפנות את הדף שהשימוש האחרון בו היה רחוק ביותר בעבר. העיה בLRU שהיא דורשת TIMESTAMP וזה מאוד לא יעיל לכן לא ממוש ומשתמשים בקירוב.
- **קירובים ל-LRU:** שימוש בסיבית שימוש (Reference Bit) שנדלקת בכל גישה. המערכת מאפסת אותה מדי פעם ובוחרת דפים שהסיבית שלהם כבויה (אלגוריתם השעון / Clock).

מדיניות דו-שלבית לבחירת תהליך: במערכות עם טבלת דפים לכל תהליך (הירכית למשל), קשה לסרוק את כל המסגרות על מנת לחפש כאלה שלא היו בשימוש לאחרונה. ומתוכן בוחרים את המסגרות לפינוי
- **לינוקס (Linux):** בוחרת את התהליך שגרם למספר חריגי הדף הקטן ביותר לאחרונה.
- **חלונות (Windows):** מדיניות working sets. לכל תהליך יש מכסה, ואם הוא חורג ממנה מפנים דפים ממנו.

אזור דפדוף (Swap) ומיפוי קבצים: אזור דפדוף: המקום בדיסק אליו מפנים דפים. בלינוקס זה יכול להיות מחיצה (partition) או קובץ, וניתן לראותם בפקודה swapon -s.

מיפוי קבצים לזיכרון (Memory Mapped Files): הקצאת זיכרון וירטואלי שממופה לקובץ שהוא כאילו מערך בזיכרון.

אני מקצה ישירות לאזור הדפדוף ואז ממפה את האזורים, שאגש פעם ראשונה לקובץ במיפוי אקבל Page fault ואז מערכת ההפעלה תביא את הדף המסגרת המתאימה מהדיסק
- **מיפוי משותף:** כתיבה לזיכרון משנה את הקובץ. משמש לתקשורת בין תהליכים ולטעינת ספריות משותפות (Shared Objects).
- **מיפוי פרטי:** כתיבה יוצרת עותק פרטי (Copy-on-Write). משמש לטעינת קוד ומשתנים.

תהליכים ותזמון מעבדים (פרק 4)

תהליכים (Processes): תהליך הוא מחשב וירטואלי שמריץ תוכנית אחת עבור משתמש אחד. תהליך יכול להיות באחד משני מצבים עיקריים:
- **רץ:** מתבצע כרגע על מעבד פיזי.
- **מושעה:** ממתין. מערכת ההפעלה משעה תהליך כאשר הוא מחכה למשאב כלשהו (למשל לבלוק נתונים מקובץ או לטיפול בחריג דף), או כאשר המערכת מחליטה שצריך להריץ תהליך אחר במקומו.

מבנה נתונים לייצוג תהליך PCB: מערכת ההפעלה מחזיקה מבנה נתונים בRAM עבור כל תהליך הכולל את השדות הבאים:
- **מצב המעבד:** ערכי האוגרים (Registers), כולל מצביע התוכנית, כדי שאפשר יהיה להחזיר תהליך מושעה לריצה מאותה נקודה בדיוק.
- **מפת זיכרון וירטואלי:** המצביע לטבלת הדפים של התהליך.
- **טבלת שגרות איתות (Signals):** רלוונטי ביוניקס/לינוקס. אלו שגרות שיופעלו כאשר מאותתים לתהליך או בזמן אירוע חריג. (בחלונות יש מנגנון שונה לטיפול בחריגים).
- **זהות והרשאות:** זהות המשתמש המריץ והרשאות הגישה שלו.
- **טבלת משאבים זמינים:** חומרה), הוא מקבל ממערכת ההפעלה "מזהה משאב" (handle בחלונות, file descriptor בלינוקס/יוניקס). המזהה הוא למעשה אינדקס בטבלת המשאבים של התהליך. הטבלה שמורה בזיכרון שנגיש רק במצב מיוחס. כל איבר בטבלה מכיל הרשאות (קריאה/כתיבה וכולי) ומצביע למבנה הנתונים האמיתי שמייצג את המשאב בזיכרון של מערכת ההפעלה. ההצבעה הכפולה הזו (אינדקס ← מצביע) היא מנגנון הגנה המונע מתהליכים לזייף מזהה משאב ולגשת למשאבים שאין להם הרשאה אליהם.
- מצב תזמון, זמן ריצה שנצבר וכו'.

מיתוג תהליכים (Context Switch): מערכת ההפעלה מעבירה את המעבד הפיזי מתהליך אחד לאחר באופן הבא:
1. המעבד עובר מקוד של התוכנית לשגרה של מערכת ההפעלה כתוצאה מקריאת מערכת או פסיקה (כולל פסיקת שעון).
2. מערכת ההפעלה מטפלת באירוע (מבצעת את השירות המבוקש או מטפלת בפסיקה).
3. מערכת ההפעלה מחליטה שצריך להעביר את המעבד לתהליך אחר.
4. המערכת שומרת את מצב המעבד הנוכחי במבנה הנתונים של התהליך המופסק כדי שניתן יהיה להמשיך את הרצתו בעתיד.
5. המערכת משחזרת את מצב המעבד של התהליך שצריך לרוץ כעת.

זהו למעשה מנגנון מיפוי של מעבדים לתהליכים, אנלוגי למנגנון המיפוי של דפים למסגרות בניהול זיכרון.

תהליכים לעומת חוטים (Threads):
- **חוט (Thread):** מעבד וירטואלי (תהליכון).
- **תהליך מרובה חוטים:** מדמה מחשב וירטואלי מרובה מעבדים.

כל החוטים של אותו תהליך משתפים את כל השדות במבנה הנתונים של התהליך (כולל מפת הזיכרון, קבצים פתוחים), פרט ל:
- **מצב המעבד:** לכל חוט אוגרים משלו.
- **מחסנית:** לכל חוט יש מחסנית משלו, אבל היא באזר הזכרון הכללי של התהליך אז לכל החוטים יש גישה אליה.
- **חלק ממצב התזמון.**

תהליכי (חוטי) גרעין (Kernel Threads): אלו תהליכים של מערכת ההפעלה עצמה, הניגשים רק למבני הנתונים שלה ולא לזיכרון של תהליך משתמש כלשהו. מבני הנתונים של מערכת ההפעלה ממופים בצורה זהה לכל התהליכים (למשל בחלק העליון של הזיכרון), ולכן תהליך גרעין יכול לרוץ עם *כל* טבלת דפים קיימת. שדה טבלת הדפים במבנה הנתונים שלהם ריק. בזמן מיתוג אליהם מערכת ההפעלה לא מחליפה טבלת דפים, מה שחוסך החלפה אחת או שתיים של טבלאות. תהליכי גרעין מועילים כאשר מעוניינים שמטלות של מערכת ההפעלה יתחרו על משאבי מעבד עם תהליכים רגילים (בלינוקס הם משמשים לפינוי מסגרות, למשל).

יצירת תהליכים: בלינוקס/יוניקס (fork): קריאת המערכת fork יוצרת תהליך חדש שהוא "תאום זהה" לתהליך שיצר אותו (התהליך הקורא), פרט למספר התהליך. הקריאה חוזרת פעמיים (פעם אחת בתהליך האב ופעם אחת בתהליך הבן). לכל אחד מהתהליכים יש מרחב זיכרון זהה אך פרטי.
- **Copy-on-Write:** כדי לייעל את השכפול ולא להעתיק את כל הזיכרון פיזית, המערכת מסמנת את הדפים כ"קריאה בלבד" בשני התהליכים.
- ניסיון כתיבה גורם לחריג דף. מערכת ההפעלה מאבחנת את הסיבה, מעתיקה את המסגרת הספציפית הזו, ומשחזרת את הרשאת הכתיבה המקורית לדף הפרטי החדש.

בחלונות (Windows): תהליך חדש תמיד מריץ תוכנית מההתחלה (אין מנגנון fork). **תזמון מעבדים (CPU Scheduling): מטרות התזמון:**
- **שימוש יעיל במעבדים פיזיים:** כדאי להמעיט במיתוג תהליכים (תקורה).
- **זמן תגובה ראוי:**
  - לתוכנית אינטראקטיבית: זמן תגובה שייצור תחושה של תגובה מיידית (בכפוף למגבלות החושיות של האדם).
  - למערכות השולטות על מערכות פיזיות (זמן אמת): זמן תגובה שמאפשר שליטה (לפני שהמטוס מתרסק).
  - לחישובים (כמו סימולציית מזג אוויר): סיום לפני הדד-ליין (מהדורת החדשות).

תוכניות שגרתיות (לא זמן אמת) אינן מכריזות על זמן תגובה נדרש, ולכן מערכת ההפעלה מסתפקת בתזמון הוגן תוך התייחסות כללית לעדיפויות.

מטרות משניות:
- **עמידה בלוחות זמנים:** קריטי במערכות זמן אמת (Real Time), אך במערכות הפעלה כלליות קשה להבטיח זאת בגלל מורכבות (למשל חריגי דף).
- **תזמון דביק (Affinity):** במחשב מרובה מעבדים, רצוי להריץ תהליך על המעבד שהריץ אותו לאחרונה, כדי למרב את התועלת מזיכרון המטמון (Cache) ומה-TLB שנשארו חמים. במחשב עם זיכרון מבוזר (NUMA), כדאי להריץ תהליך על מעבד שקרוב פיזית למסגרות הזיכרון של התהליך.
- **תזמון כנופיות (Gang Scheduling):** כשמריצים תוכנית מרובת חוטים, כדאי לתזמן את הרצת כל החוטים של התהליך בבת אחת על מעבדים שונים, כדי לאפשר תקשורת יעילה ביניהם.

מבני נתונים לתזמון: תורים רב-רמתיים: המבנה הנפוץ הוא **Multilevel Feedback Queues**: ישנם מספר תורים (למשל תור עליון בעדיפות מקסימלית, תור תחתון בעדיפות מינימלית). לכל תור מוקצית "פרוסת זמן" (Time Slice) שונה. הכלל הבסיסי: המעבד תמיד לוקח את התהליך הראשון מהתור הגבוה ביותר שאינו ריק.

סיום קוונטה: אם תהליך מגיע לסוף הקוונטה ולא הסתיים הוא עובד לסוף התור אבל באותה הרמה (ללא קריאת I/O)

כללים למעבר בין תורים (Feedback): המבנה כולל כללים דינמיים למעבר תהליכים בין התורים, המבוססים על התנהגות התהליך:
- **ירידה:** תהליך שצבר זמן ריצה רב שמוגדר בחוקים של הMLFQ זה יכול להיות גדול הקוונטה ויכול ליהיות משהו אחר עובר לתור נמוך יותר.
- **עלייה (מניעת הרעבה):** תהליך שלא רץ זמן רב עובר לתור גבוה יותר, כדי למנוע הרעבה (Starvation).
- **קבלת משאב:** תהליך שחיכה למשאב (כגון דיסק) וקיבל אותו, נכנס לתור גבוה מאוד. המטרה: שירוץ מיד, ישחרר את המשאב מהר ככל האפשר, ויחזור להמתין. זה משפר את נצילות ה-I/O.

העדיפות שאנו מעניקים לתהליכים (למשל עדיפות "גבוהה" שהמשתמש קובע) משפיעה על הפרמטרים של מעבר בין התורים, ולא בהכרח קובעת תור ספציפי קבוע לתהליך. השיטה מאפשרת לממש מדיניות הוגנת ויעילה, וגם יכולות מיוחדות (כגון תהליכים שרצים רק כשאף אחד אחר לא רוצה לרוץ).

תכנות תהליכים בו-זמניים (פרק 5)

מטרות בתכנות בו-זמני (Concurrent Programming):
- **ניצול חומרה:** רצון לנצל מספר מעבדים פיזיים במחשב (מקביליות).
- **הסתרת זמני המתנה:** רצון לנצל את המעבד בזמן שהתוכנית ממתינה להתקן איטי (כגון קריאה מהדיסק).
- **חווית משתמש:** מתן אשליה למשתמש שמספר פעולות קורות במקביל (למשל, טיפול במקלדת/עכבר בזמן שהתוכנית מציגה אנימציה או משמיעה מוזיקה), גם אם יש מעבד יחיד.
- **שרתים:** מימוש שרתים שבהם כל לקוח מטופל על ידי חוט נפרד.
- **מודולריות:** הפרדת מודולים שיכולים לרוץ בסדר ביצוע משתנה (מודולריזציה טמפורלית).

תיאום (Coordination): כאשר חוטים שונים רצים במקביל, כל חוט מבצע פעולות משלו. אם הפעולות בלתי תלויות, ניתן להשתמש בגישת "שגר ושכח". אולם ברוב המקרים, יש לכפות אילוצים על סדר הפעולות כדי להבטיח נכונות.

אם החוטים מתזמנים כך ששניהם קוראים את אותו ערך התחלתי לפני שהראשון כותב, שניהם יכתבו את אותו ערך והמשתנה יגדל ב-1 בלבד במקום ב-2. זוהי "מרוץ תהליכים" (Race Condition).

מנעולים (Locks / Mutexes): מנעול הוא אובייקט שמבטיח **מניעה הדדית** (Mutual Exclusion). למנעול שני מצבים: **חופשי** (Free) **ונעול** (Locked).
- lock(m): אם המנעול חופשי, הפעולה נועלת אותו וחוזרת מיד. אם הוא נעול, החוט המבקש נכנס להמתנה עד שהמנעול ישתחרר.
- unlock(m): משחרר את המנעול. לחוט מותר לשחרר רק מנעול שהוא עצמו נעל.

חוט שמחזיק במנעול מונע מחוטים אחרים להיכנס ל"קטע הקריטי" המוגן על ידי המנעול.

על מה מגן המנעול? המנעול אינו מגן טכנית על "מבנה הנתונים" (הזיכרון), אלא על ה**אינווריאנטה** (Invariant - תכונה שחייבת להישמר) של מבנה הנתונים. לדוגמה: ברשימה מקושרת, האינווריאנטה היא שהמצביעים תקינים ומחברים את האיברים ברצף. בזמן הוספת איבר, האינווריאנטה מופרת זמנית. המנעול מונע מחוטים אחרים לראות את המבנה במצב הלא-תקין הזה.

משתני תנאי / אירועים (Condition Variables): מנגנון שמאפשר לחוט אחד להמתין להודעה מחוט אחר (למשל: "החוצץ אינו ריק"). במערכות POSIX (לינוקס/יוניקס), הפעולות הן:
- wait(c, m): החוט משחרר את המנעול m ונכנס להמתנה על משתנה התנאי c באופן אטומי. כשהחוט מתעורר, הוא נועל מחדש את המנעול m לפני שהוא חוזר מהפונקציה.
- signal(c): מעיר חוט יחיד שממתין על c.
- broadcast(c): מעיר את **כל** החוטים שממתינים על c.

בחלונות (Windows) המנגנון שונה ונקרא Events (אירועים), עם סוגים כמו Manual-Reset ו--Auto-Reset.

דגשים במימוש:
- **שימוש ב-while ולא ב-if:** כאשר חוט מתעורר מ-wait, הוא חייב לבדוק שוב את התנאי (למשל, האם החוצץ באמת לא מלא?). ייתכן שחוט אחר תפס את המנעול לפניו ושינה את המצב, או שהתרחשה "התעוררות סרק" (Spurious Wakeup).
- **שימוש ב-broadcast:** אם יש מספר קוראים ומספר כותבים, שימוש ב-signal עלול להעיר חוט מהסוג הלא נכון (למשל, כותב מעיר כותב אחר כשהחוצץ מלא), מה שעלול להוביל לקיפאון. broadcast פותר זאת (במחיר יעילות) או שימוש בשני משתני תנאי נפרדים (אחד ל"לא מלא" ואחד ל"לא ריק").

קיפאון (Deadlock): מצב שבו שני חוטים (או יותר) ממתינים זה לזה לנצח, כך שאף אחד לא יכול להתקדם. דוגמה קלאסית:
- חוט א' נועל את $m1$ ומנסה לנעול את $m2$.
- חוט ב' נועל את $m2$ ומנסה לנעול את $m1$.

שני החוטים נתקעים. במודל פורמלי של גרף משאבים, קיפאון מתבטא **במעגל** בגרף.

התמודדות עם קיפאון:
- **הימנעות (Avoidance):** הגדרת סדר גלובלי על המשאבים וחיוב כל החוטים לנעול משאבים לפי הסדר הזה (למשל, תמיד לנעול לפי סדר הכתובות בזיכרון). זה מונע מעגלים.
- **היחלצות/גילוי:** המערכת מזהה מעגל בגרף המשאבים ו"הורגת" את אחד החוטים. מקובל בבסיסי נתונים, אך בעייתי במערכות הפעלה כלליות (התוכנית תתרסק).
- **מניעה:** הגבלת זמן המתנה למנעול (קיים בחלונות).

גרעיניות (Granularity): הבחירה כמה מידע מנעול אחד מגן עליו:
- **גרעיניות גסה (Coarse-grained):** מנעול אחד לכל מבנה הנתונים (למשל, לרשימה שלמה). פשוט למימוש, אך רק חוט אחד יכול לעבוד על הרשימה בכל רגע, מה שפוגע במקביליות.
- **גרעיניות עדינה (Fine-grained):** מנעול לכל איבר ברשימה. מאפשר למספר חוטים לעבוד על חלקים שונים של הרשימה במקביל, אך התקורה (Overhead) של הנעילות והשחרורים גדולה יותר, והמימוש מורכב יותר.

הגינות (Fairness) והרעבה: מנעולים רגילים אינם מבטיחים הגינות (מי שמחכה הכי הרבה זמן לא בהכרח יכנס ראשון). במצבי עומס, זה עלול לגרום ל**הרעבה** (Starvation) של חוטים מסוימים. דוגמה: **מנעול קוראים/כותבים**. מותר להרבה קוראים לקרוא במקביל, אך כותב דורש בלעדיות. מימוש נאיבי שנותן עדיפות לקוראים עלול לגרום לכך שכותב לא יקבל גישה לעולם אם תמיד מגיעים קוראים חדשים. פתרון הוגן דורש מנגנון "כרטיסים" או תור מסודר.

היפוך עדיפויות (Priority Inversion): בעיה במערכות זמן אמת עם תזמון מבוסס עדיפויות:
1. חוט בעדיפות נמוכה (L) נועל משאב.
2. חוט בעדיפות גבוהה (H) צריך את המשאב ונכנס להמתנה.
3. חוט בעדיפות בינונית (M) נכנס למערכת ומשתלט על המעבד (כי העדיפות שלו גבוהה מ-L, ו-L מחזיק את המשאב).

התוצאה: החוט הגבוה (H) ממתין בעקיפין לחוט הבינוני (M), למרות שיש לו עדיפות עליו. **פתרון:** הורשת עדיפויות (Priority Inheritance) - חוט שמחזיק משאב יורש זמנית את העדיפות הגבוהה ביותר של מי שממתין לו.

מימוש מנעולים בחומרה: לא ניתן לממש מנעולים יעילים עם פקודות קריאה/כתיבה רגילות בלבד (במיוחד בריבוי מעבדים). החומרה מספקת פקודות אטומיות מיוחדות, כגון: **Compare & Swap (CAS):** פקודה שמקבלת כתובת, ערך ישן וערך חדש. היא מעדכנת את הערך רק אם הערך הנוכחי בזיכרון שווה לערך הישן, וכל זה בפעולה אטומית אחת שלא ניתנת להפרעה. פקודות אלו מאפשרות גם לממש מבני נתונים ללא מנעולים (Wait-free / Lock-free), כמו מונה אטומי, שבהם חוט שתקוע לא תוקע את שאר המערכת.

מערכות קבצים (פרק 6)

הגדרות ומטרות: מערכת הקבצים (File System) מספקת שני מנגנונים עיקריים:
1. **מרחב שמות (Name Space):** מיפוי שמות הירככיים וקריאים לבני אדם (מחרוזות) לעצמים של מערכת ההפעלה ולמיקומם הפיזי. דוגמאות:
   /usr/bin/ls,
   /dev/lp0,
   C:\windows\system32\kernel32.dll.
2. **אחסון ואחזור (Storage & Retrieval):** מנגנון לשמירה של נתונים על התקנים חיצוניים (בעיקר דיסקים, כרטיסי זיכרון, ותקליטורים) וקריאתם בחזרה.

הערה: במערכות מסוימות (מקינטוש, חלונות) קובץ יכול להיות מורכב ממספר "רצפים" (Streams), אך נתעלם מאפשרות זו כעת.

מרחב השמות (Name Space): מרחב השמות מאורגן כעץ או גרף חסר מעגלים עם שורש.
- **צמתים פנימיים:** מדריכים/ספריות/תיקיות (Directories).
- **עלים:** קבצים או עצמים אחרים (התקנים, צינורות, ערוצי תקשורת).
- **מצביע סימבולי:** עלה מסוג מיוחד המצביע לעצם אחר.
- **שמות:** ביוניקס/לינוקס השמות ניתנים ל**קשתות** בגרף. ההתייחסות לעצם היא על ידי שרשור שמות הקשתות לאורך המסלול מהשורש.

מרחב השמות בחלונות (Windows): בניגוד ליוניקס, בחלונות לעצמים עצמם יש שמות (ולא לקשתות):
- לעצם יכולים להיות מספר שמות נרדפים (למשל, תמיכה בשמות קצרים בסגנון DOS של 8.3 תווים).
- מרחב השמות הוא אחוד וכולל גם עצמים השמורים בדיסק וגם **עצמים נדיפים** של המערכת (כמו מנעולים, אירועים, וזיכרון משותף).
- שמות קבצים רגילים מתפענחים תחת \??\, ועצמים נדיפים תחת \BaseNamedObjects\.

מצביעים סימבוליים (Symbolic Links): עלים המכילים שם של עצם אחר:
- כאשר הפענוח מגיע למצביע סימבולי, תוכן המצביע משורשר ל**סיומת** של השם המקורי שטרם פוענח.
- הפענוח מתחיל מחדש עם השם החדש שנוצר.
- **דוגמה:** אם /lib/xyz הוא קישור ל-/etc, אז בפענוח /lib/xyz/passwd, המערכת תחליף את הרישא ותתחיל לפענח את /etc/passwd.
- ניתן ליצור מצביע סימבולי גם אם היעד לא קיים (קישור שבור).
- **איך מוודאים שלא יצרנו מעגל?** התשובה היא שלא מוודאים

נקודות הצבה (Mount Points): מנגנון לחיבור מערכות קבצים נפרדות לעץ אחד:
- מצביע מצומת במרחב השמות לשורש של מרחב שמות אחר (למשל מדיסק אחר).
- כאשר הפענוח מגיע לנקודת הצבה, הוא ממשיך עם יתרת השם בתוך המערכת החדשה.
- **מימוש:** ביוניקס - טבלה נדיפה בזיכרון. בחלונות - "נקודות פענוח" (Reparse Points) בדיסק, שיכולות להפעיל שגרה שממשיכה את הפענוח.

מעגלים ומחיקת קבצים: מערכת ההפעלה מונעת יצירת מעגלים בגרף (למשל, אוסרת hard link למדריך).
- **הסיבה:** מניעת מעגלים מאפשרת שימוש במונה הצבעות (Reference Counting) לניהול זיכרון. אם היו מעגלים, חלקים מנותקים היו נשארים עם מונה חיובי והיה צורך ב-Garbage Collector יקר.
- **מניעת קובץ בודד:** בגלל שמערכת ההפעלה מונעת מעגלים, יצירת מונה מצביעים לכל קובץ ומוחקת עצמים שמונה המצביעים שלהם הגיע ל0
- תוכניות (כמו גיבוי) צריכות להיזהר ממעגלים שנוצרים ע"י קישורים סימבוליים.

סמנטיקה של גישה לקבצים:
- **עקביות (Consistency):** גישה ממספר תהליכים.
  - מודל Sequential Consistency: כתיבה לקובץ נראית "מיד" לתהליכים אחרים. כל קריאה רואה את הכתיבה האחרונה, גם אם המידע טרם נכתב לדיסק הפיזי. הפעולות אטומיות (לא רואים מידע חלקי).
- **עמידות (Durability/Persistence):** עמידות בפני נפילות.
  - כתיבה רגילה **אינה** מבטיחה עמידות מיידית (המידע נשמר בזיכרון המטמון).
  - סגירת קובץ או שימוש בפעולות מיוחדות (fsync) מבטיחות כתיבה לדיסק, אך פוגעות בביצועים.

מיפוי קבצים לדיסק: מערכת ההפעלה מחלקת את הקובץ לבלוקים בגודל קבוע. האתגר: מיפוי הבלוקים הלוגיים לבלוקים פיזיים בדיסק.
- **הקצאה רציפה:** מפשטת מיפוי ומשפרת ביצועים, אך גורמת ל**פיצול חיצוני** (External Fragmentation) הרבה חורים,קטנים מידי לקובץ גדול ולא מאפשרת הגדלת קובץ ללא הזזה אם אין אחריו חור.
- **בלוקים גדולים:** מקטינים את התקורה של המיפוי ואת גודל טבלאות המיפוי ומשפרים את ביצועי הדיסק, אך גורמים ל**פיצול פנימי** (בזבוז מקום בבלוק האחרון של הקובץ).

שיטות מיפוי נפוצות:
1. **יוניקס/לינוקס (Inodes):** מבנה ה-Inode מכיל מצביעים ישירים, ומצביעים עקיפים (יחיד, כפול, משולש) לקבצים גדולים. מטא-דאטה (בעלים, הרשאות, גודל) נשמר ב-Inode.
2. **חלונות (NTFS):** כל קובץ מיוצג ע"י רשומה בטבלה ראשית (MFT - Master File Table). קבצים קטנים נשמרים בתוך הרשומה עצמה. קבצים גדולים מיוצגים ע"י "הרצות" (Runs) - רצפי בלוקים רציפים.
3. **מערכת FAT:** טבלה אחת גדולה בתחילת הדיסק, המשמשת כרשימה מקושרת של בלוקים. יעיל לדיסקים קטנים, אך גישה אקראית איטית (מעבר סדרתי על הרשימה).

מדיניות הקצאת בלוקים: המטרה: למקם קבצים בצורה שתמזער את תנועת ראש הדיסק.
- רצוי למפות רצפים לוגיים ארוכים לרצפים פיזיים.
- רצוי למקם קבצים מאותה ספרייה קרוב זה לזה (Locality).

FFS (Fast File System): מערכת שפותחה ב-BSD Unix לשיפור ביצועים:
- **קבוצות גלילים (Cylinder Groups):** הדיסק מחולק לקבוצות. כל קבוצה מכילה עותק של ה-Superblock, מאגר Inodes ומפת ביטים משלה לייצוג הבלוקים הפנויים במפת הצילנדרים.
- **מדיניות הקצאה:**
  - בחירת קבוצת גלילים:
    * אם הקובץ קיים ובקבוצת צלינדרים שבו הוא מוקצה יש מקום, והקובץ תופס פחות מרבע ממנה, נקצה באותה קבוצה, אחרת נמצא קבוצה אחרת עם תפוסה נמוכה
    * אם הקובץ חדש, נקצה בקבוצה שמכילה את המדריך שבו הקובץ
  - בחירת בלוק:
    * הבלוק הקרוב ביותר מבחינה סיבובית של הדיסק לבלוק האחרון בקובץ, אם הוא פנוי, אחרת משתמשים במדיניות משנית
- **אשכולות:** הקצאה ביחידות של 64KB כדי לשפר קצב העברה.

LFS (Log-Structured File System): גישה המבוססת על ההנחה שכתיבות הן צוואר הבקבוק (כי קריאות מגיעות מהמטמון).
- **הרעיון:** כל הדיסק הוא יומן (Log) אחד גדול הדיסק מחולק למקטעים בגודל קבוע שמשורשרים לרשימה.
- **כתיבה:** כל המידע (נתונים + מטא-דאטה + Inodes) נכתב תמיד לסוף היומן באופן סדרתי. אין שכתוב במקום (In-place).
- **שכתוב:** עדכון בלוק גורם לכתיבתו מחדש בסוף היומן, ולעדכון ה-Inode שיצביע למיקום החדש (שגם הוא נכתב לסוף).
- **ניקוי (Cleaning):** כדי לפנות מקום, תהליך רקע קורא מקטעים (Segments) ישנים, מזהה אילו בלוקים עדיין "חיים" (עדכניים), מעתיק אותם לסוף היומן, ומסמן את המקטע הישן כריק.
  - כתיבות יעילות מאוד:
    * תמיד לבלוקים רצופים בדיסק
    * בדרך כלל מקטעים שלמים או חלקים גדולים שלהם
  - קריאות מבלוקים שרירותיים בדיסק, אבל אם כתיבה ביחד מנבאת קריאה ביחד, אז קריאות מאזורים קרובים בדיסק
  - ביצועים מצוינים כאשר ביצועי כתיבה חשובים יותר מביצועי קריאה או כאשר יש יותר כתיבות מקריאות (קבצי לוג, למשל)
  - ביצועים גרועים במערכות מרובות משתמשים: כתיבות בו-זמניות לא מנבאות דבר לגבי קריאות
  - רעיון שקם לתחייה במערכות אחסון מבוססות זיכרון הבזק כי זול בהן לכתוב ברצף פיזי ויקר לשכתב in place

נפילות והתאוששות (Crash & Recovery): השגרות של מערכת הקבצים מניחות הנחות מסוימות אודות מבנה הנתונים על הדיסק. נכונות השגרות תלויה בקיום ההנחות הללו (אינווריאנטות - Invariants - שנשמרות). נפילה פתאומית עלולה להשאיר את המערכת במצב לא תקין שבו ההנחות מופרות.

הנחות שחייבות להתקיים (קריטיות):
- בלוק שהוא חלק מקובץ אינו מסומן כפנוי (אחרת הוא יידרס).
- בלוק אינו ממופה כחלק משני קבצים או יותר (אחרת שינוי באחד ישחית את השני).
- מצביע במדריך מצביע ל-Inode שאינו מסומן כפנוי.
- ועוד...

הנחות פחות חשובות שניתן לתקן בזמן שהמערכת פועלת:
- בלוק שאינו ממופה כחלק מקובץ וגם אינו מסומן כפנוי ("דליפת מקום").
- Inode שאין אליו מצביע ואינו מסומן כפנוי.

גישות להתאוששות: זיהוי הצורך בהתאוששות: הדלקת סיבית מיוחדת ("סיבית מלוכלכת") במערכת הקבצים מכריזה שהמערכת ירדה באופן מסודר והיא קונסיסטנטית. הכתיבה ה**ראשונה** למערכת הקבצים בעת העלייה היא כיבוי הסיבית הזו (סימונה כ"לא נקייה"). אם בעת אתחול המערכת הסיבית כבויה, סימן שהמערכת לא ירדה בצורה מסודרת ודרושה התאוששות.

1. כתיבה זהירה (Careful Writes): פעולות שמשנות את מבנה הנתונים בדיסק מתבצעות מיידית (לפני שקריאת המערכת חוזרת) והבלוקים נכתבים לדיסק **בסדר שמבטיח** שלא ייווצר חוסר קונסיסטנטיות חמור.

דוגמה - סדר פעולות במחיקת קובץ:
1. מחיקת המצביע ל-Inode מהמדריך.
2. מחיקת ההצבעות לבלוקים מתוך ה-Inode.
3. סימון ה-Inode והבלוקים כפנויים במפות הביטים.

(סדר זה מבטיח שאם המערכת נופלת באמצע, לכל היותר יהיו בלוקים תפוסים שאינם שייכים לאף קובץ, אך לא יקרה שבלוק של קובץ אחר יידרס).
- הגישה משמשת את FAT, FFS, ו-LFS.
- **חיסרון בביצועים:** גישה זו משפיעה לרעה על הביצועים כאשר קבצים נוצרים ונמחקים באופן תכוף (כי היא מחייבת כתיבות סינכרוניות רבות). ב-LFS ההשפעה פחותה כי הכתיבות רצופות.
- אחרי נפילה, תהליך התאוששות (fsck) רץ ברקע ומתקן את הבעיות הלא חמורות (כמו סימון בלוקים "יתומים" כפנויים).

2. עדכונים רכים (Soft Updates): מבנה נתונים בזיכרון שומר את כל השינויים שצריך לבצע בדיסק. לא משנים ישירות את העותקים של הבלוקים בזיכרון המטמון.
- לגבי כל שינוי, המערכת זוכרת את השינויים ש**חייבים לבצע לפניו** כדי לא ליצור מצב לא קונסיסטנטי חמור.
- זהו למעשה גרף שבו השינויים הם צמתים והתלויות הן קשתות.
- כאשר צריך לכתוב לדיסק, מערכת הקבצים מזהה את כל השינויים שצריך לכתוב ואת כל מי שצריך לכתוב לפניהם, מסדרת אותם, משנה את העותקים בזיכרון וכותבת לדיסק.
- **כתיבות כפולות:** יתכנו בלוקים שצריך לכתוב יותר מפעם אחת, בגלל שהם מכילים כמה שינויים שביניהם צריך לכתוב שינויים לבלוקים אחרים.
- למרות זאת, ניסויים הראו שהביצועים טובים הרבה יותר מכתיבה זהירה, מכיוון שניתן לעכב בזיכרון עדכונים רבים ולבצע אופטימיזציות.

3. כתיבה עצלה (Lazy Writes): מבצעים שינויים על עותקים של בלוקים בזיכרון וכותבים אותם לדיסק לפי סדר שנוח מבחינת תזמון הדיסק (למשל, כדי למזער תנועת ראש). מתעלמים מבעיית הקונסיסטנטיות.
- כאשר המערכת עולה אחרי נפילה, יש צורך להריץ תהליך התאוששות (fsck) שסורק את כל הדיסק ומתקן את מבנה הנתונים.
- **לפני שהתהליך מסיים לא ניתן להשתמש במערכת הקבצים**, כיוון שהנחות בסיסיות על מבנה הנתונים עלולות שלא להתקיים.
- גישה זו הייתה בשימוש בעבר בלינוקס (למשל ext2), אך אינה נפוצה במערכות הפעלה מסחריות מודרניות.
- **יתרון:** ביצועים מצוינים בשגרה.
- **חיסרון:** התאוששות איטית מאוד (עד שעות במערכות קבצים גדולות).

4. מערכות מתאוששות (Recoverable Systems): שימוש בטכנולוגיה של מסדי נתונים (יומן אירועים - Journaling) על מנת לוודא שפעולות שמשנות את מבנה הנתונים ודורשות שינוי במספר בלוקים מתבצעות במלואן או לא מתבצעות בכלל (אטומיות), גם אם מתרחשת נפילה.
- כל פעולה כזו מוגדרת כ**תנועה** (Transaction) ומקבלת מזהה.
- כל שינוי בבלוק של מבנה הנתונים (שימו לב: לא בבלוק נתונים של קובץ, אלא במטא-דאטה!) נרשם ב**קובץ יומן מיוחד** עם מזהה התנועה.
- בסיום כל פעולה, נרשמת רשומה מיוחדת ביומן עם מזהה התנועה (Commit).
- מערכת הקבצים לא מבצעת את השינויים בבלוקים המקוריים מיד, וגם לא כותבת לדיסק את רשומות היומן מיד, אבל היא מוודאת שבלוק נכתב לדיסק רק **אחרי** שכל רשומות היומן שרלוונטיות אליו נכתבו בבטחה (Write-Ahead Logging).

מצב אחרי נפילה: אחרי נפילה, מערכת הקבצים לא קונסיסטנטית ואי אפשר להשתמש בה לפני הרצת תהליך התאוששות (כמו בכתיבה עצלה, לא כמו בזהירה).
- ייתכן שיש בדיסק שינויים ששייכים לתנועות שאולי לא בוצעו במלואן, ואולי אפילו לא נכתבו ליומן בדיסק במלואן.
- ייתכן שחסרים בדיסק שינויים ששייכים לתנועות שמתוארות ביומן בדיסק במלואן (שהסתיימו בהצלחה לוגית אך לא פיזית).

תהליך ההתאוששות (Log Replay):
1. קריאת היומן מהדיסק, הולכים אחורה עד הCHECKPOINT האחרון ואז הולכים קדימה, יש סימנים לפתיחת טרנזקציה וסגירתה נעזרים בהם כדי לאפיין למה צריך לעשות undo ולמה redo.
2. **ביטול (Undo):** זיהוי כל התנועות שנכתבו בחלקן אבל לא במלואן ליומן. בעזרת רשומות היומן מחזירים את הבלוקים הרלוונטיים למצבם הקודם.
3. **ביצוע מחדש (Redo):** זיהוי כל התנועות שנכתבו ליומן במלואן (Committed) אבל אולי לא בוצעו מול הדיסק במלואן. בעזרת רשומות היומן מבצעים שוב את השינויים בבלוקים הרלוונטיים.

הערה חשובה: ייתכן שהשינויים כבר בוצעו בפועל לפני הנפילה; אי אפשר לדעת. לכן, רשומות היומן חייבות לתאר **עדכונים אידמפוטנטיים** (Idempotent). כלומר, ביצוע הפעולה פעם אחת או מאה פעמים מביא לאותה תוצאה בדיוק.

סיכום מערכות מתאוששות:
- **התאוששות מהירה:** צריך לבצע שוב או לבטל רק תנועות מהשניות האחרונות לפני הנפילה (ולא לסרוק את כל הדיסק).
- **ביצועים מצוינים:** יש מעט אילוצים על סדר הכתיבה (רק האילוץ שהיומן נכתב לפני הבלוק).
- **מורכבות:** המערכות מורכבות מאוד למימוש בגלל הצורך בזיהוי מהיר ומדויק של תנועות, כפיית אילוצים, והתמודדות עם מצבי קצה (כמו נפילה נוספת בזמן ההתאוששות עצמה).
- מערכות אלו נפוצות מאוד כיום: חלונות (NTFS), רוב הגרסאות המסחריות של יוניקס, ולינוקס (ext3, ext4, XFS).

5. שימוש בזיכרון לא נדיף (NVRAM): אחסון בלוקים של מבנה הנתונים (ואולי אף בלוקים של נתונים מקבצים) בזיכרון מיוחד שאינו נדיף, במקום בזיכרון הראשי הנדיף.
- לאחר נפילה, המידע עדיין שם, ופשוט כותבים את הבלוקים הללו לדיסק.
- הזיכרון הזה צריך לשרוד לא רק תקלות חומרה (הפסקות חשמל), אלא גם **תקלות תוכנה** שעלולות לכתוב לתוכו בטעות. לכן עדיף להגן עליו מכתיבה רוב הזמן ולשחרר את ההגנה רק בתוך שגרות קריטיות של מערכת הקבצים.
- ניתן למימוש בעזרת זיכרון רגיל המגובה בסוללה (Battery Backed RAM) או אל-פסק, או בעזרת טכנולוגיות זיכרון מגנטי מהיר.
- נפוץ בשרתי קבצים ייעודיים (Appliances), אך פחות במערכות הפעלה רגילות למחשב אישי.

מערכות קבצים וזיכרונות הבזק (Flash): בדרך כלל זיכרונות הבזק מנוהלים על ידי בקר חומרה ייעודי שגורם להם להיראות למערכת ההפעלה כמו דיסק מגנטי רגיל (אפשר לקרוא ולכתוב בלוקים קטנים לפי כתובת). בפועל, הבקר ממפה כתובות לוגיות לפיזיות.
- **בהתקנים יקרים (SSD):** המיפוי הפנימי דומה מאוד ל-Log Structured File System (כתיבה למקומות חדשים וניקוי זבל).
- **בהתקנים זולים (Disk-on-Key):** יש טבלת מיפוי בגרעיניות גסה. כתיבות אקראיות הן איטיות מאוד כי הן גורמות להעתקת הרבה נתונים (Read-Modify-Write) של בלוקים גדולים).
- משתמשים במערכות קבצים רגילות, עם התאמות קלות (כמו פקודת Trim לסימון בלוק שנמחק, כדי שהבקר יידע שלא צריך להעתיק אותו בניקוי הזבל).
- כאשר זיכרון ההבזק חשוף ישירות למעבד (ללא בקר מסתיר), משתמשים במערכות קבצים מיוחדות (FFS, JFFS2) המבוססות בדרך כלל על רעיונות של LFS.

רשתות תקשורת ו-IP (פרק 7)

הרצוי לעומת המצוי: קיים פער משמעותי בין צרכי התוכנה לבין יכולות החומרה:
- **ברמת היישום:** דרוש קשר אמין בין תוכניות שרצות על מחשבים שונים שאינם מחוברים ישירות זה לזה, אלא בעקיפין דרך רשת תקשורת מורכבת (כמו האינטרנט).
- **ברמת החומרה:**
  - ניתן לתקשר רק בין מחשבים שמחוברים פיזית (ישירות).
  - התקשורת לא תמיד אמינה (רעשים, ניתוקים).
  - התקשורת מתבצעת בדרך כלל במנות (packets), שהן חבילות מידע בדידות בגודל קבוע או מוגבל.

מודל השכבות (Layers): על הפער מגשרים באמצעות חלוקה לשכבות, כאשר כל שכבה מספקת שירותים לשכבה שמעליה ומשתמשת בשירותי השכבה שמתחתיה:
1. **השכבה הפיזית (Physical):** המאפיינים הפיזיים של ערוץ התקשורת. מגדירה רמות מתח/אור, סוג מחבר, כבל, תדרים וכדומה. דוגמה: מודם.
2. **שכבת המקשר (Link):** מגדירה פרוטוקולים להעברת מנות מידע בין מחשבים שמחוברים פיזית בתווך משותף. הפרוטוקול מגדיר את מבנה המנה, שיטת מיעון (Addressing) ועוד. דוגמאות: Ethernet, PPP.
3. **שכבת הרשת (Network):** מגדירה פרוטוקולים לניתוב מנות בין מחשבים ברשת שאינם מחוברים בהכרח פיזית זה לזה. הפרוטוקול המרכזי הוא IP.
4. **שכבת ההעברה (Transport):** מגדירה פרוטוקולים לתקשורת בין תהליכים ספציפיים במחשבים שונים. דוגמאות: TCP, UDP.
5. **שכבת היישום (Application):** מגדירה פרוטוקולים ליישומים ספציפיים. דוגמאות: העברת קבצים (FTP), גלישה (HTTP), דואר אלקטרוני (SMTP).

שכבת המקשר: אתרנט (Ethernet): פרוטוקול נפוץ לחיבור מחשבים ברשת מקומית (LAN).
- **מבנה:** מקטע (Segment) המחבר מספר מחשבים (למשל בבניין יחיד, עקב מגבלת מרחק).
- **מיעון:** לכל מחשב (כרטיס רשת) יש כתובת אתרנט ייחודית (MAC Address).
- **תקשורת:** כל מחשב יכול לתקשר עם כל אחד מהמחשבים האחרים במקטע.
- **מנגנון שידור:** ניתן לקלוט רק אם מחשב אחד בלבד משדר ברגע נתון. הפרוטוקול כולל מנגנון לזיהוי התנגשויות (CSMA/CD).
- **קצב:** במקור 10Mb/s. כיום 1Gb/s עד 10Gb/s (ועד 400 בסיב אופטי).

אתרנט ממותג (Switched Ethernet): חידוש (שהיום הוא הסטנדרט) המשתמש ברכיב חומרה שנקרא **מתג** (Switch).
- המתג לומד איזו כתובת אתרנט מחוברת לכל כניסה (פורט) שלו.
- המתג מעביר כל מנה רק לפורט של היעד שלה (ולא לכולם).
- מאפשר תקשורת בו-זמנית בקצב מלא בין זוגות שונים של מחשבים (מונע התנגשויות).
- מבחינת המחשבים, אין הבדל בין מקטע רגיל לממותג.

אתרנט אלחוטי (WiFi):
- מבנה לוגי דומה למקטע אתרנט: כל התחנות שומעות זו את זו.
- **מורכבות:** אי אפשר להבטיח שמשדר יזהה תמיד התנגשות (בעיית התחנה הנסתרת - Hidden Terminal Problem).
- **אבטחה:** קל לצותת לשידורים באוויר, ולכן אבטחת מידע וגישה קריטיות יותר. מנגנונים ישנים (WEP) היו קלים לפריצה.
- קצבים: מ-1MB/s ועד מעל 1GB/s, תלוי באיכות הקליטה ובחומרה.

טכנולוגיות רשת נוספות:
- **רשתות סלולריות:** LTE, 5G.
- **סיבים אופטיים:** GPON, EPON (סיב עד הבית).
- **קווי נחושת:** ADSL/VDSL (תקשורת ספרתית על קו טלפון אנלוגי, קצב לא סימטרי).
- **כבלים:** מודמים לתשתית טלוויזיה בכבלים (DOCSIS).
- טכנולוגיות שנעלמו: Token-Ring, Decnet, מודמים אנלוגיים (Dial-up).

שכבת הרשת: פרוטוקול IP: הרשת בנויה כהיפר-גרף שבו המחשבים הם צמתים, וקשתות (רשתות מיקשר) מחברות ביניהם.
- **כתובות IPv4:** לקצוות של קשתות יש שמות בני 32 סיביות (למשל 132.67.110.201). עקרונית הכתובות ייחודיות בעולם.
- **נתב (Router):** מחשב המחובר למספר רשתות ותפקידו להעביר מנות ביניהן.

עקרונות הניתוב (Routing): המנה "מפלסת" את דרכה ברשת מהמקור ליעד. המסלול אינו נקבע מראש.
- כל מחשב/נתב במסלול מחליט מהי הקשת הבאה שהמנה תעבור בה, ומי המחשב הבא (Next Hop) אליו תועבר המנה.
- ההחלטה מתבצעת על פי **כתובת היעד** המצוינת במנה ובעזרת **טבלת ניתוב**.

תהליך הניתוב וההחלטה (Routing Decision & Flow): המטרה: העברת פקטה מכתובת מקור לכתובת יעד. ה-IP נשאר קבוע, ה-MAC משתנה בכל קפיצה.

1. אלגוריתם ההחלטה (Longest Prefix Match - LPM): כיצד הנתב בוחר את השורה הנכונה בטבלה?
- **הבדיקה:** עבור כל שורה, מבצעים Bitwise AND בין כתובת היעד ל-Subnet Mask. אם התוצאה זהה ל-Network Destination, השורה מתאימה.
- **ההכרעה:** אם נמצאו מספר שורות מתאימות, בוחרים תמיד בשורה בעלת ה-**Mask הארוך ביותר** (מספר ה-'1' הגדול ביותר / ה-CIDR הגבוה ביותר).
- **כלל אצבע:** נתיב ספציפי (/24) תמיד מנצח נתיב כללי (/16), שמנצח את (/0) Default Gateway.

1.1 חישוב טווח מהיר (טריק Magic Number): כיצד למצוא את גבולות ה-Subnet ללא המרה לבינארית?
- **1. מצא את הבלוק:** באוקטטה המשתנה (שאינה 0 או 255), בצע:

$$\text{Block Size} = 256 - \text{Mask Value}$$

- **2. הגדרת הטווח:** הרשת מתחילה בכתובת הבסיס, ומסתיימת ב: $(\text{Base} + \text{Block} - 1)$.
- **דוגמה (עבור Mask .192):**
  החישוב: $256 - 192 =$ **64**.
  אם הכתובת היא 128,. הטווח הוא 128 עד **191** $(128 + 64 - 1)$.

2. זרימת המנה ליעד מרוחק (Packet Flow): דוגמה: מחשב שולח ל-20.20.20.20 (אוסטרליה).
- **שלב א' - המקור (Source Host):** בדיקה בטבלה מגלה שהיעד אינו ברשת המקומית.
  פעולה: שימוש ב-Default Gateway. המנה נשלחת ל-MAC של הראוטר המקומי.
- **שלב ב' - בדרך (Intermediate / ISP):** הנתב מוצא התאמה (למשל לפי LPM) לנתב הבא (Next Hop).
  פעולה: הקטנת TTL, החלפת כתובת **MAC** (שלו → של הנתב הבא), ושליחה. ה-IP לא משתנה!
- **שלב ג' - נתב היעד (Last Hop):** הנתב מוצא בטבלה שורת רשת המוגדרת כ-**On-link** (מחובר פיזית).
  פעולה: ביצוע **ARP Request** כדי לגלות את ה-MAC של מחשב היעד, ומסירה ישירה.

תחזוקת טבלאות הניתוב:
- **במחשב אישי:** בדרך כלל עדכון אוטומטי בהתחברות לרשת באמצעות פרוטוקול DHCP שמקצה כתובת ומשער ברירת מחדל (Default Gateway). ניתן גם לערוך ידנית (route).
- **בנתבים:** שימוש באלגוריתמים מבוזרים ודינמיים (BGP) הלומדים את מבנה הרשת ומוצאים מסלולים קצרים.
- חשוב למנוע **מעגלי ניתוב** (מנה שמסתובבת לנצח מבזבזת משאבים).

תרגום כתובות (NAT): מכיוון שאין מספיק כתובות IPv4 ציבוריות לכל המכשירים בעולם:
- משתמשים בכתובות פרטיות (Private Addresses) בתוך ארגונים ובתים (למשל 10.0.0.0/8 או 192.168.0.0/16). כתובות אלו אינן מנותבות באינטרנט הציבורי.
- הנתב (Router/Gateway) שמחבר את הרשת לאינטרנט מבצע תרגום (Network Address Translation). הוא מחליף את הכתובת הפרטית במנה היוצאת לכתובת הציבורית שלו ומקצה מספר שער (port) מדומה כדי לזכור למי שייכת התשובה.

פרוטוקול IPv6: נועד לפתור את מצוקת הכתובות ולתת כתובת ציבורית לכל רכיב (כולל IoT).
- כתובות באורך **128 סיביות** (במקום 32).
- סימון: 8 קבוצות של 16 סיביות בבסיס הקסדצימלי, מופרדות בנקודתיים.
- ניתן להשמיט אפסים מובילים ורצפי אפסים (ע"י ::). דוגמה: 2001:db8::1.
- סוגי כתובות:
  - Link-local: תקפות רק למקטע הנוכחי (fe80::/10).
  - Global unicast: כתובות ציבוריות רגילות (2000::/3).
  - Unique local: מקביל לכתובות פרטיות ב-IPv4.
  - Multicast: מחליף את ה-Broadcast (ff00::/8).

שכבת ההעברה: שקעים (Sockets): כתובת IP מציינת מחשב. כדי ליצור קשר בין **תהליכים**

ספציפיים, צריך כתובת לתהליך.
- **שקע (Socket):** עצם תוכנה שדרכו שולחים ומקבלים מידע.
- **שער (Port):** הכתובת של השקע (מספר בן 16 סיביות). שרתים מאזינים בשערים ידועים (למשל HTTP=80).

פרוטוקולים עיקריים:
- **UDP:** שליחת מנדעים (Datagrams) - מעין "מברקים" בודדים. לא אמין (שגר ושכח).
- **TCP:** חיבור אמין וקבוע (Connection) בין שני שקעים. מעביר רצף בתים (Stream) ומבטיח הגעה וסדר נכון.

זרימת יצירת קשר (TCP Handshake):
1. **שרת:** יוצר שקע, קושר אותו לשער (bind), ומכריז על האזנה (listen). ואז מחכה (accept).
2. **לקוח:** יוצר שקע ומבקש להתחבר לכתובת ושער של השרת (connect).
3. השרת מקבל את הבקשה, ו-accept יוצרת **שקע חדש** ייעודי לשיחה עם הלקוח הספציפי הזה (כדי שהשקע המקורי יחזור להאזין לאחרים).

זרימת מידע במחסנית הפרוטוקולים: בעת שליחת מידע:
1. **יישום:** כותב מידע לשקע.
2. **העברה (TCP/UDP):** מפרקת למנות, מוסיפה תחילית (Header) עם מספרי שערים.
3. **רשת (IP):** מוסיפה תחילית עם כתובות IP (מקור ויעד) ומחליטה על הניתוב.
4. **מיקשר (Link):** מוסיפה תחילית עם כתובות פיזיות (MAC) ושולחת לחומרה.

פרוטוקול ARP/NDP: תרגום כתובות רשת (IP) לכתובות פיזיות (MAC).
- מחשב שרוצה לשלוח מנה לכתובת IP שנמצאת באותו מקטע, שולח שאלת **Broadcast**: "למי יש את הכתובת X?".
- המחשב בעל הכתובת עונה: "לי, וזו הכתובת הפיזית שלי".
- התשובה נשמרת בזיכרון מטמון (ARP Table) כדי לחסוך שאלות עתידיות.
- ב-IPv6 הפרוטוקול נקרא NDP (במקום ARP).

פרוטוקול TCP (פרק 8)

הפרוטוקול מספק מענה לאתגרים הבאים:
- שונות בגודל המנות ברשתות שונות.
- תקלות שידור (Transmission Failures).
- שמירה על סדר (Sequencing).
- בקרת זרימה (Flow Control).
- בדיקת שגיאות מקצה לקצה.
- יצירה והריסה של קשרים לוגיים בין תהליכים.

המודל מניח קיום של תהליכים במחשבים (Hosts) שונים שרוצים לתקשר זה עם זה.

תכונות הפרוטוקול: TCP הוא פרוטוקול ההעברה הנפוץ ביותר בזכות התכונות הבאות:

1. קשר בין תהליכים: הקשר הוא בין תהליכים רצים במחשבים שונים (ולא רק בין כרטיסי רשת).
- שימוש ב-Sockets (שקעים): עצם שדרכו שולחים ומקבלים מידע.
- כתובת השקע נקראת **שער** (Port). דוגמאות: שער 80 (ווב), שער 3068.
- יצירה ופירוק של הקשר נעשים באופן מפורש ומוסכם על ידי שני הצדדים (Handshake).

2. אופי הקשר:
- **דו-כיווני (Full Duplex):** מידע זורם לשני הכיוונים באופן עצמאי.
- **סדור (Ordered):** המידע מתקבל בצד השני בדיוק בסדר שבו נשלח. זאת בניגוד ל-IP שמעביר מנות ולא רצף, ולא מבטיח סדר.
- **אמין (Reliable):** מובטח שהמידע יגיע. פרוטוקול IP אינו אמין וחבילות עלולות לא להגיע כלל.

3. יעילות:
- העברה מהירה ככל האפשר של מידע.
- מניעת עומס מיותר על הרשת (למשל, עומס שנובע ממשלוח חוזר של מנות שאבדו).

האתגר: מימוש מעל רשת לא אמינה: המימוש מורכב מכיוון שפרוטוקול הרשת (IP) אינו אמין ואינו סדור:
- נתב עמוס פשוט מוחק מנות (למשל במעבר מערוץ מהיר לאיטי).
- שינויים בטבלאות הניתוב גורמים לשינוי מסלולים, כך שמנה שנשלחה מאוחר עשויה לעקוף מנה שנשלחה מוקדם.

עקרונות המימוש (נגזרות מהדרישות): כדי לגשר על הפער בין הרשת הלא-אמינה לדרישות ה-TCP:
1. **רצף סדור:** מחייב **מספור סודר** (Sequence Number) על המנות, כדי שהמקבל יוכל לשחזר את הסדר המקורי.
2. **אמינות:** מחייבת **משלוח חוזר** (Retransmission) של מנות שאבדו. זה גורר צורך ב:
   - **חוצצים (1):** השולח חייב לשמור עותק של המידע בחוצץ עד שיודע שהוא התקבל (כי התהליך השולח לא יודע על אובדן המידע).
   - **חוצצים (2):** המקבל צריך חוצץ כי התהליך "מושך" מידע בקצב שלו, ואי אפשר "לדחוף" לו מידע.
   - טיפול בכפילויות: מנות עלולות להגיע פעמיים (אם מנה התעכבה וחשבו שאבדה).
3. **יעילות (שליחה חוזרת לא אובססיבית):** מחייבת **אישורי קבלה** (ACKs) כדי לזהות מנות שלא הגיעו תוך זמן סביר ולשלוח רק אותן שוב.

מבנה החוצצים והחלונות: תפקיד החוצצים:
- **בצד השולח:** מאפשר לשלוח כמות גדולה של מידע לפני קבלת אישור למנה הראשונה (ניצול רוחב פס).
- **בצד המקבל:** מאפשר קליטת מידע גם כשהתהליך המקבל עסוק.

מבנה תחילית המנה (TCP Header): לכל מנה מצורפים השדות הבאים:
- **כמות המידע:** מספר הבתים במנה.
- **מספר סודר (Sequence Number):** המיקום של המידע ברצף הכללי (מודולו 2^{32}).
- **מספר אישור (Acknowledgment Number):** מיקום הבית האחרון שנקלט **ללא חורים** בצד השולח את האישור. (לא ניתן לאשר קבלת מנה אם יש "חור" לפניה).
- **גודל חלון (Window Size):** מודיע לצד השני כמה מקום פנוי נשאר בחוצץ הקבלה. זהו מנגנון בקרת הזרימה.

חלונות מחליקים (Sliding Windows): זהו המנגנון המרכזי לניהול השידור:
- **מבט השולח:** החלון מגדיר את הנתונים שנשלחו אך טרם אושרו, ואת הנתונים שמותר לשלוח כעת. כאשר מתקבל אישור (ACK), החלון "זז ימינה" ומאפשר שליחת נתונים חדשים.
- **מבט המקבל:** החלון מגדיר את התחום שהוא מוכן לקלוט (בהתאם למקום הפנוי בחוצץ).

תרחיש לדוגמה:
1. מתקבלת מנה שממלאת "חור" ברצף הנתונים אצל המקבל.
2. המקבל שולח אישור (ACK) מעודכן המצביע על סוף הרצף הרציף החדש.
3. השולח מקבל את האישור ומזיז את החלון שלו ימינה.
4. במקביל, כשהתהליך המקבל קורא נתונים מהחוצץ, המקבל שולח **עדכון חלון** (Window Update) שמגדיל את גודל החלון המותר לשליחה.

טיפול באובדן מנות: מנגנון Timeout אחראי על שידור חוזר:
- שליחת מנה מפעילה טיימר ("יקיצה").
- אם הטיימר פוקע לפני הגעת אישור, שולחים שוב את המנה הישנה ביותר שלא אושרה.
- **הרחקה אקספוננציאלית:** בכל כישלון, פרק הזמן ליקיצה גדל פי קבוע (עם תקרה), כדי למנוע הצפת רשת איטית בשידורים חוזרים.
- כאשר מגיע אישור, הטיימר מתאפס ומתוזמן מחדש למנה הבאה בתור.

טיפול באובדן אישורים (מנגנון Persist): אם אישור (ACK) הולך לאיבוד, השולח עלול לחשוב שהחלון של המקבל סגור (גודל 0) ולהפסיק לשלוח. **הפתרון:** שירות יקיצה בשם Persist.
- מופעל כאשר יש מידע לשלוח אך החלון סגור.
- השולח שולח מנת בדיקה ("גישוש") קטנה (למשל בית בודד).
- המקבל עונה עם אישור המכיל את גודל החלון העדכני.
- המקבל מאשר גם מידע שכבר התקבל בעבר, למקרה שהאישור המקורי אבד.

אופטימיזציות בפרוטוקול:

סינדרום החלון האווילי (Silly Window Syndrome): מצב שבו נשלחות מנות קטנות מאוד ולא יעילות (כי התקורה של הכותרות גדולה מהמידע). זה קורה כשהחוצץ בצד המקבל כמעט מלא, והוא מתפנה לאט (בית אחרי בית). **הפתרון:**
- **המקבל:** אינו מכריז על פתיחת חלון אלא אם כן התפנה מקום למנה סטנדרטית שלמה (או חצי גודל חוצץ). עד אז הוא מכריז על חלון 0.
- **השולח:** משהה משלוח מנות קטנות. שולח רק אם יש לו מנה מלאה, או אם החלון שנפתח הוא לפחות חצי מהחלון המקסימלי.

האלגוריתם של Nagle: מטפל במצב שבו השולח יוצר מידע בקצב איטי (למשל הקלדה ב-Telnet). **הכלל:** השולח משהה משלוח של מידע חדש שאינו ממלא מנה שלמה, כל עוד יש מנות קודמות שנשלחו וטרם אושרו (Unacked).
- ברוחב פס גבוה (רשת מקומית) האישורים מגיעים מהר ולכן אין עיכוב מורגש.
- ביישומים אינטראקטיביים מסוימים (משחקי רשת) זה מפריע, וניתן לכיבוי ע"י setsockopt (דגל TCP_NODELAY).

השהיית אישורים (Delayed ACKs): מטרה: למנוע משלוח מנות ריקות שרק מאשרות קבלה ("תקורת רשת"). מנה שאינה נושאת מידע אלא רק ACK ועדכון חלון מעוכבת עד ש:
1. נוצר צורך לשלוח נתונים (ואז ה-ACK מצורף עליהם - Piggybacking).
2. או שחלון הקבלה גדל למנה שלמה.
3. או שחלף זמן קצוב (בדרך כלל 200 מילי-שניות), כדי לא לגרום ל-Timeout בצד השני.

מניעת עומס (Congestion Avoidance): עומס בנתבים גורם לאובדן מנות. השולח מזהה יקיצות (Timeouts) כסימן לעומס ברשת (ולא סתם כרעש בקו). **התגובה:** הקטנת קצב השליחה ("חלון העומס") כדי לאפשר לרשת להתאושש.

שליחה חוזרת מהירה (Fast Retransmit): אם השולח מקבל **3 אישורים זהים** ברצף (Duplicate ACKs), הוא מסיק שמנה בודדת אבדה (ושהמנות שאחריה הגיעו ולכן המקבל חוזר על האישור האחרון התקין). במקרה כזה, שולחים שוב את המנה האבודה **מיד**, מבלי לחכות ל-Timeout, ואינם מקטינים את קצב השליחה דרסטית (כי הרשת עדיין מעבירה מידע).

קביעת גודל חוצצים (Buffer Sizing): תפקיד החוצץ הוא לאפשר שליחה רציפה ללא המתנה לאישורים. גודל החוצץ האופטימלי הוא:

Bandwidth × RTT (Round Trip Time)

התחלה איטית (Slow Start): בתחילת הקשר, השולח לא יודע מה קיבולת הרשת. שליחת חלון מלא מיד עלולה להפיל נתבים. **הפתרון:**
- מתחילים עם "חלון עומס" (Congestion Window) בגודל מנה אחת.
- על כל אישור שמתקבל, מגדילים את החלון ב-1.
- התוצאה: קצב השליחה גדל באופן **אקספוננציאלי** עד שמגיעים ליציבות או לאובדן מנה.

ניהול מחזור חיי הקשר: יצירת קשר (Three-Way Handshake):
1. **לקוח:** שולח SYN עם מספר סידורי אקראי x.
2. **שרת:** עונה SYN עם מספר סידורי אקראי y, ו-ACK המאשר את x (מבקש $x+1$).
3. **לקוח:** שולח ACK המאשר את y (מבקש $y+1$).

השימוש במספרים אקראיים נועד להקשות על התחזות ולמנוע בלבול עם קשרים ישנים.

פירוק קשר (Connection Termination): הפירוק הוא הדרגתי (כל צד סוגר את הכיוון שלו):
1. צד א' שולח FIN.
2. צד ב' שולח ACK. (כעת צד א' לא שולח יותר, אך יכול לקלוט).
3. צד ב' שולח את שארית המידע שלו, ולבסוף שולח FIN.
4. צד א' שולח ACK.

טיפול בקשרים סגורים:
- **מנות תועות:** מנות שהתעכבו עלולות להגיע לאחר שהקשר נסגר. מערכת ההפעלה שומרת את מבנה הנתונים של הקשר לזמן מה (TIME_WAIT) כדי לטפל בהן ולמנוע פתיחת קשר חדש עם אותם פרמטרים שיתבלבל מהמנות הישנות. ניתן לעקוף זאת עם setsockopt.
- **מנות לקשר לא קיים:** אם מגיעה מנה לקשר שנשכח או לא היה קיים, המערכת עונה במנה עם דגל **RST** (Reset) הגורמת לצד השני להפסיק לשלוח מיד.

מודעות לקיום קשר (Keepalive): כאשר אין מידע להעביר, TCP שקט לחלוטין. זה גורם לחוסר מודעות אם הצד השני נפל או הכבל נותק. ניתן להפעיל מנגנון Keepalive השולח מנות ריקות מדי פעם כדי לוודא שהצד השני עדיין חי.

Structure: TCP/IP Full

IP[Version, IHL, TOS, TotalLen, ID, IPFlags(DF,MF), FragOffset, TTL, Proto=6, HeaderChecksum, SrcIP, DestIP, Options, TCP[SrcPort, DestPort, SeqNum, AckNum, DataOffset, Reserved, TCPFlags(CWR,ECE,URG,ACK,PSH,RST,SYN,FIN), Window, Checksum, UrgPtr, Options, Data]]

Structure: UDP/IP Full

IP[Version, IHL, TOS, TotalLen, ID, IPFlags(DF,MF), FragOffset, TTL, Proto=17, HeaderChecksum, SrcIP, DestIP, Options, UDP[SrcPort, DestPort, Length, Checksum, Data]]

מערכות קבצים מבוזרות (פרק 9)

מבוא והגדרות: מערכת מבוזרת (Distributed System) היא קבוצה של מחשבים וציוד היקפי (כמו מדפסות) המחוברים ברשת תקשורת (בתוך ארגון, בחדר שרתים, או בענן). המטרה היא לאפשר למשתמשים לנצל חומרה דרך הרשת ולא רק חומרה מקומית.

למה לא מערכת הפעלה אחת? מדוע לא מריצים מערכת הפעלה אחת ענקית ששולטת על כל החומרה המבוזרת?
- **טיפול בתקלות חלקיות:** במחשב יחיד, או שהכל עובד או שהכל כבוי/מקולקל. במערכת מבוזרת, ייתכן שחלק מהרכיבים (מחשבים/דיסקים) תקולים, ועדיין רוצים להשתמש בשאר המערכת.
- **אבטחה:** במחשב יחיד, מערכת ההפעלה שולטת בהכל. במערכת מבוזרת, הודעות זדוניות עלולות להסתנן לרשת מבחוץ.
- **ביצועים:** יש להתחשב באיטיות של רשת התקשורת (השהיות ורוחב פס).

מערכות קבצים מבוזרות (DFS): מערכות אלו מאפשרות למחשבים ("לקוחות") לגשת לקבצים המאוחסנים במחשב אחר ("שרת"). **יתרונות:**
- **פיזור עומס:** ניתן לפזר עומס חישובי בין לקוחות שונים שעובדים על מידע משותף.
- **גישה גיאוגרפית:** גישה לקבצים ממיקומים שונים (מחדרים שונים בארגון ועד יבשות שונות, למשל Google Drive).
- **ניהול מרכזי:** קל יותר לנהל ולגבות קבצים בשרת מרכזי אחד מאשר בהרבה תחנות קצה.
- **עלויות:** לעיתים זול יותר לרכוש דיסקים לשרת מרכזי (שיתוף משאבים) מאשר דיסק לכל תחנה (אם כי לפעמים דיסקים לשרתים יקרים יותר).

מרחב השמות (Name Space): כיצד הלקוח רואה את הקבצים המרוחקים?

גישת יוניקס (Unix): הצבה (Mounting): מערכת הקבצים המרוחקת "מוצבת" על גבי עץ התיקיות המקומי בנקודה שרירותית.
- בשרת: יש מערכת קבצים (למשל תחת /export).
- בלקוח: מבצעים mount של התיקייה המרוחקת לתוך תיקייה מקומית (למשל /home/cs/).
- **שקיפות:** המשתמש לא בהכרח יודע שהקבצים נמצאים בשרת אחר.

גישת חלונות (Windows):
- בעבר: שימוש באותיות כונן (למשל Z:) שמופו לשרתים.
- כיום (וגם בעבר): שימוש בשמות שרתים מפורשים (UNC Paths). שם הקובץ מכיל את שם השרת. דוגמה: \\fs.x.org\homes\stoledo. השרת מגדיר "שם שיתוף" (Share Name) לתיקיות שהוא חושף.
- החל מחלונות 2000 ניתן גם לבצע הצבה (Mount) בנקודה שרירותית כמו ביוניקס.

גישות נוספות:
- **AFS (Andrew File System):** כל הקבצים בעולם נמצאים תחת עץ גלובלי אחד, למשל /afs/ibm.com/user/...
- **URL:** גישה לקבצים באינטרנט (HTTP, FTP) או דואר אלקטרוני (IMAP).

השוואה: שמות מפורשים מול הצבה שרירותית:
- **שם משקף שרת:** המשתמש יודע לאן הוא ניגש, שם הקובץ קבוע ואבסולוטי. חסרון: אם השרת מוחלף, השם משתנה וצריך לעדכן את המשתמשים.
- **הצבה שרירותית:** גמישות, ניתן להחליף שרת בלי שהמשתמש ירגיש. חסרון: דורש ניהול (מנהל מערכת צריך להגדיר את ההצבות), ועלול ליצור בלבול אם במחשבים שונים ההצבה נעשית במקומות שונים.

פענוח שמות מרוחקים: מערכת ההפעלה של הלקוח מזהה שהנתיב המבוקש נמצא תחת נקודת הצבה מרוחקת (או מתחיל בשם שרת). הלקוח שולח את **יתרת השם** (הסיומת) לשרת לפענות. השרת מחזיר "מזהה קובץ" (File Handle).

הטמנה (Caching): האתגר הגדול במערכות מבוזרות הוא הפער בין מהירות הרשת למהירות הדיסק.
- רשתות (כמו 100Mb/s או 1Gb/s) לרוב איטיות יותר מקצב העברת נתונים של דיסק מקומי או מערך דיסקים בשרת.
- בנוסף, השרת מטפל בהרבה לקוחות במקביל (עומס).

הפתרון: הלקוח שומר עותקים של נתונים (בלוקים) בזיכרון המטמון המקומי שלו, כדי לחסוך פניות לשרת לקריאה חוזרת, וכדי לאחד כתיבות ("חציצה").

בעיות סמנטיקה (Semantics): ההטמנה יוצרת בעיות עקביות (Consistency):
1. **קריאה מיושנת:** לקוח א' שינה בלוק ושמר במטמון שלו (טרם נשלח לשרת). לקוח ב' קורא את הבלוק מהשרת ומקבל נתונים ישנים.
2. **עדכון אבוד:** לקוח א' כותב לשרת. לקוח ב' קורא מאוחר יותר, אבל משתמש בעותק ישן שנמצא במטמון **שלו** (של ב'), ולא פונה לשרת.
3. **ערבוב נתונים:** קריאה של קובץ גדול עלולה להחזיר חלק מהבלוקים מעודכנים (כי הם לא היו במטמון ונקראו מהשרת) וחלק ישנים (כי הם היו במטמון).

פתרונות לבעיית הסמנטיקה:
1. **ללא הטמנה:** כל גישה עוברת לשרת. פתרון בטוח אך איטי מאוד (כמעט לא בשימוש).
2. **סמנטיקה של "מושב" (Session Semantics - AFS):**
   - כשפותחים קובץ, הוא מועתק בשלמותו ללקוח.
   - כל הפעולות מתבצעות מקומית.
   - בסגירה, הקובץ המעודכן נשלח חזרה לשרת.
   - **יתרון:** ביצועים טובים. **חסרון:** שינויים נראים לאחרים רק בסגירת הקובץ.
3. **NFS:** התעלמות חלקית מהבעיה לטובת ביצועים ופשטות (פירוט בהמשך).

מערכת NFS (Network File System): פרוטוקול שפותח ע"י Sun Microsystems. העקרונות המרכזיים:

פרוטוקול חסר מצב (Stateless): השרת **אינו זוכר** שום מידע על הלקוחות (אילו קבצים פתוחים אצל מי, מי קורא מה). הוא רק יודע למי מותר לגשת (בקרת גישה).
- **יתרון בשרידות:** אם לקוח נופל, השרת לא נפגע (אין "זבל" לנקות). אם השרת נופל ועולה, הלקוחות יכולים להמשיך לעבוד כאילו לא קרה כלום (כי אין לשרת זיכרון שאבד).

פתיחת קובץ ומזהה (File Handle): כשלוקח מבקש לפתוח קובץ (שולח שם), השרת מחזיר **מזהה** (Handle) המכיל את מיקום הקובץ בדיסק (Inode Number) ואת זמן יצירתו (Generation Number).
- המזהה תקף לתמיד (כל עוד הקובץ קיים), גם אם השרת עשה Reboot.
- הלקוח משתמש במזהה זה לכל הקריאות והכתיבות הבאות.

פעולות קריאה/כתיבה:
- הלקוח מטמין בלוקים. אם הבלוק במטמון הוא "טרי" (פחות מ-30 שניות, למשל), משתמשים בו. אם הוא ישן, שואלים את השרת אם הקובץ השתנה.
- כתיבות נשלחות לשרת באופן אסינכרוני (ברקע), אבל חובה לשלוח את כולן לפני שהפקודה close מסתיימת (כדי להבטיח שמירה). אין צורך לשלוח הודעת "סגירה" לשרת.

אמינות ואידמפוטנטיות (Idempotency): NFS רץ במקור מעל UDP (לא אמין). הלקוח אחראי לוודא ביצוע (שידורים חוזרים אם אין תשובה).
- השרת עלול לקבל את אותה בקשה פעמיים (אם האישור המקורי אבד).
- לכן, בקשות צריכות להיות **אידמפוטנטיות** (ביצוע חוזר לא משנה את התוצאה).
- דוגמה: "כתוב נתונים X להיסט Y" היא אידמפוטנטית. "תקרא את הבלוק הבא" אינה אידמפוטנטית (כי המיקום משתנה), לכן ב-NFS תמיד מציינים היסט מפורש בקריאה.

פעולות לא אידמפוטנטיות: יש פעולות בעייתיות, כמו יצירת קובץ (open עם O_CREAT | O_EXCL), שאמורה להיכשל אם הקובץ קיים.
- אם הבקשה הראשונה הצליחה (יצרה קובץ) אך האישור אבד, הבקשה החוזרת תיכשל (כי הקובץ כבר קיים). הלקוח יחשוב בטעות שהקובץ היה קיים קודם לכן.
- פתרון: הלקוח מנסה לברר פרטים על הקובץ הקיים כדי לנחש אם הוא זה שיצר אותו הרגע (פתרון יוריסטי ולא מושלם).

ביצועי כתיבה (NFS V3): כתיבה סינכרונית לדיסק של השרת היא איטית.
- בגרסה 3, הלקוח יכול לשלוח כתיבות לשרת ולהרשות לו לשמור אותן בזיכרון (RAM) בלבד.
- השרת מחזיר אישור זמני.
- בשלב מאוחר יותר (למשל בסגירת הקובץ), הלקוח שולח פקודת Commit. רק אז השרת חייב לכתוב לדיסק ולדווח.
- אם השרת נופל לפני ה-Commit, הלקוח יודע שעליו לשלוח את הנתונים שוב.

סוגי הצבות ב-NFS: מה קורה כשהשרת לא עונה?
- **הצבה קשיחה (Hard Mount):** הלקוח לא מוותר לעולם. התהליך נתקע (Hang) עד שהשרת חוזר. מתאים למערכות קריטיות או עיבוד אצווה (Batch).
- **הצבה רכה (Soft Mount):** לאחר זמן קצוב (Time-out) קריאת המערכת חוזרת עם שגיאה. מתאים לסביבה אינטראקטיבית, אך התוכנית צריכה לדעת לטפל בשגיאות אלו (למשל, לשמור עותק מקומי). עלול לגרום לאובדן מידע שקט אם לא מטופל נכון.
- **הצבה ניתנת לפסיקה (Interruptible):** הלקוח מחכה, אבל המשתמש יכול להרוג את התהליך (למשל Ctrl-C).

מערכת NQNFS וחוזים (Leases): שיפור ל-NFS שמוסיף "מצב" (State) כדי לשפר עקביות והטמנה.
- הלקוח מבקש מהשרת **"חוזה שכירות"** (Lease) לזמן קצוב על קובץ.
- **חוזה קריאה:** הלקוח רשאי להטמין את הקובץ ולקרוא ממנו מקומית. השרת מבטיח שאף אחד לא כותב לקובץ בזמן זה.
- **חוזה כתיבה:** הלקוח רשאי לכתוב לקובץ ולעדכן את המטמון שלו. השרת מבטיח שהוא הלקוח היחיד שניגש לקובץ.

ביטול חוזים: אם לקוח אחר רוצה לגשת לקובץ, השרת פונה ללקוח המחזיק בחוזה ומבקש לבטלו ("Callback"). הלקוח חייב לרוקן את המטמון (או לשלוח כתיבות) ולבטל את החוזה. **התאוששות:** אם התקשורת נותקה, השרת פשוט מחכה שהחוזה יפוג (Time-out) ואז הוא חופשי לתת חוזה למישהו אחר. זהו "מצב רך" שמותר לשכוח אותו אחרי זמן מה.

הגנה ואבטחה (פרק 10)

מבוא: סכנות ומנגנוני הגנה: הסכנות האורבות למערכות מחשב מתחלקות לשני סוגים עיקריים:
- **פריצה (Break-in):** מצב שבו מי שאינו משתמש לגיטימי מצליח לבצע פעולות במערכת, או מצב שבו משתמש לגיטימי מצליח לבצע פעולות שאינו מורשה לבצע.
- **תקיפה (Attack):** פעולה של גורם חיצוני שנועדה לגרום נזק למערכת מבלי להשיג תועלת ישירה (כגון גניבת מידע).

מערכת ההפעלה מספקת מנגנונים להתמודדות:

אימות זהות (Authentication) - מי שם?, **הרשאות (Authorization/Permissions)** - איזה פעולות מותרות לכל משתמש/ת (Access Control), **מנגנוני רישום (Logging)** - תיעוד של מי עשה מה, **והגברת הרשאות והתחזות מותרת**.

נזקים אפשריים מפריצות:

קריאת מידע לא מורשית (כגון טופס מבחן או מידע פיננסי), **שינוי או הוספת מידע** (כגון שינוי ציון, יתרה בבנק, שינוי סיסמאות גישה, מפתחות הצפנה או שינוי הרשאות), **מחיקת מידע** (מחיקת תיק פלילי, עדויות לעבירה, או עדויות לפריצה למערכת המחשב "ניקוי עקבות"), **מניעת שירות (Denial of Service)** למשל הפלת אתר אינטרנט, **שימוש במשאבים ללא תשלום** (גניבת כוח חישוב לכריית מטבעות או שימוש במדפסת), **וביצוע פעולות פיזיות** במערכות מחשב השולטות על ציוד פיזי (כגון הפסקת שירות ניטור רפואי או שיבוש תקשורת).

דרכי פריצה:
- **גישה פיזית לחומרה:** גניבת המחשב או הדיסקים, ציתות לקו תקשורת (חוטי או אלחוטי), או חדירה לחדר שרתים שבו המסוף פתוח ללא אימות.
- **התחזות (Impersonation):** גניבת זהות של משתמש לגיטימי. מבחינת המערכת, מי שעובר את אימות הזהות (למשל מקיש סיסמה נכונה) הוא המשתמש הלגיטימי.
- **הרצת קוד זדוני (Malware):** תוכנית הגורמת למשתמש לגיטימי לבצע פעולות שמותרות לו, אך מבלי שהוא מודע לכך. פעולות אלו משרתות את הפורץ (למשל שליחת נתונים החוצה) או מזיקות למשתמש.
- **ניצול שגיאות בקוד (Exploits):** שליחת קלט זדוני הגורם לתוכנה רצה (או למערכת ההפעלה עצמה) לפעול באופן לא מתוכנן ולבצע את רצון התוקף.

קוד זדוני (Malware): סוגים נפוצים:
- **סוס טרויאני (Trojan Horse):** תוכנה הנראית תמימה אך מכילה מנגנון נסתר לביצוע פעולות זדוניות.
- **וירוס (Virus):** קטע קוד הנצמד לתוכנה קיימת ומשכפל את עצמו כאשר התוכנה רצה.
- **תולעת (Worm):** תוכנה עצמאית המשכפלת את עצמה דרך רשתות מחשבים.

מינוח מקצועי:
- **Bug:** שגיאה בתוכנה.
- **Vulnerability:** חולשה שאפשר לנצל.
- **Exploit:** קוד המנצל את החולשה כדי להשתלט על התוכנית.
- **Shellcode:** קוד זדוני (לרוב קצר) שהתוקף מריץ.
- **Rootkit:** אוסף כלי תוכנה המאפשרים לתוקף לשמור על שליטה במחשב ("Own") ולהסתיר את עקבותיו.
- **Botnet:** רשת של מחשבים נגועים הנשלטים מרחוק.

דוגמה לפגיעות: גלישת חוצץ (Buffer Overflow): שימוש בפונקציות לא בטוחות כמו gets המעתיקות קלט לתוך משתנה מקומי במחסנית ללא בדיקת גודל. תוקף יכול להכניס קלט ארוך מהצפוי, לדרוס את כתובת החזרה (Return Address) ששמורה במחסנית, ולגרום למעבד לקפוץ לכתובת המכילה קוד זדוני שהושתל בקלט עצמו. זה מאפשר הרצת קוד שרירותי עם הרשאות התוכנית המותקפת.

התגוננות מקוד זדוני:
- הימנעות מתוכנות ממקור לא ידוע. תוכנות קוד פתוח פופולריות נחשבות בטוחות יותר כי הקוד נבדק על ידי רבים.
- שימוש בחתימות דיגיטליות (Digital Signatures) לווידוא מקור התוכנה.
- הרצת שרתים (כמו HTTP) עם הרשאות מינימליות.
- שימוש בתוכנות אנטי-וירוס ו-EDR המזהות חתימות של קוד זדוני או דפוסי התנהגות חשודים.
- ביקורת (Auditing) על גישה לקבצים ותעבורת רשת.

הקטנת מעטפת החשיפה (Attack Surface):
- **חומת אש (Firewall):** חוסמת גישה מבחוץ לשירותים ברשת המקומית שעלולים להיות פגיעים.
  - **חסר מצב (Stateless):** מסנן לפי כתובות ושערים (Ports) מוגדרים מראש.
  - **בעל מצב (Stateful):** עוקב אחרי מצב הקישור (TCP connection) ומאפשר תעבורה רק ששייכת לקישור קיים ומורשה.
- ההנחה היא שהרשת הפנימית בטוחה יותר, ולכן שירותים פגיעים זמינים בה בחופשיות. להגנה בתוך הרשת משתמשים ב-NAC (Network Access Control).

תקיפות מניעת שירות (DoS - Denial of Service): שימוש אינטנסיבי במשאבים במטרה להשבית שירות או להאט אותו משמעותית עבור משתמשים לגיטימיים.
- תוקף שירותים הפתוחים לכל (כמו שרתי ווב או דואר).
- הצפה בבקשות גורמת לקריסת השרת או למיצוי משאבים (כמו מקום בדיסק עקב יומני רישום ענקיים).
- קשה למניעה כי הבקשות נראות לגיטימיות.
- **DDoS (Distributed DoS):** תקיפה מתואמת ממספר רב של מחשבים נגועים (Zombies/Bots) המופעלים מרחוק בו-זמנית.

אימות זהות (Authentication): זיהוי המשתמש מבוסס על שלושה גורמים אפשריים:
1. **משהו שאתה יודע:** סיסמה.
2. **משהו שיש לך:** תעודת זהות, כרטיס חכם, טלפון נייד.
3. **משהו שהוא אתה:** ביומטריה.

סיסמאות (Passwords): אמצעי הזיהוי הנפוץ ביותר.
- **בעיות:** אנשים שוכחים סיסמאות נדירות, רושמים סיסמאות מורכבות על פתקים, או בוחרים סיסמאות קלות לניחוש (שמות, תאריכים).
- **גניבה:** ניחוש, בדיקה אוטומטית (Brute Force), ציתות לקו תקשורת, גניבת קובץ הסיסמאות, או התחזות לתוכנת ההתחברות (Phishing/Spoofing).
- **הגנה:**
  – שמירת הסיסמאות כפלט של פונקציה חד-כיוונית (One-Way Hash) ולא כטקסט קריא.
  – הגנה על קובץ הסיסמאות (/etc/shadow ביוניקס).
  – שימוש בנתיב בטוח (Trusted Path) כמו Ctrl-Alt-Del בחלונות שמבטיח שמסך ההתחברות הוא של מערכת ההפעלה.
  – אכיפת מדיניות סיסמאות חזקות והחלפה תדירה (אם כי זה עלול לגרום למשתמשים לרשום אותן).

הרשאות (Permissions / Authorization): לאחר הזיהוי, המערכת קובעת מה מותר למשתמש לעשות. עצמים מוגנים: קבצים, מדריכים, שקעים, התקנים, תהליכים. פעולות: קריאה, כתיבה, הרצה, הריגת תהליך, נעילה.

מטריצת הגישה (Access Matrix): מודל תיאורטי שבו השורות הן המשתמשים והעמודות הן האובייקטים. כל תא מגדיר את הזכויות. בפועל המטריצה דלילה מאוד ולכן לא נשמרת כך, אלא בשתי שיטות:
1. **לפי עמודות - רשימות בקרת גישה (ACL - Access Control List):** לכל אובייקט מוצמדת רשימה של מי מורשה לגשת ומה מותר לו.
2. **לפי שורות - יכולות (Capabilities):** לכל תהליך יש רשימה של "כרטיסי כניסה" (מפתחות) לאובייקטים אליהם הוא מורשה לגשת.

דחיסת רשימות בקרת גישה (ACL):
- **קבוצות (Groups):** מתן הרשאה לקבוצת משתמשים במקום לכל אחד בנפרד.
- **רשימות הרשה/סרב (Allow/Deny):** הרשימה נסרקת לפי סדר. הרשומה הראשונה שתואמת למשתמש קובעת. זה מאפשר לקבוע כללים כלליים ויוצאים מן הכלל (למשל: הרשה לכולם, סרב למשתמש X).

מימוש במערכות ספציפיות:
- **יוניקס/לינוקס:** שיטה פשוטה של 3 קבוצות בלבד לכל קובץ: **בעלים** (User), **קבוצה** (Group), **אחרים** (Others). לכל אחת מוגדרות 3 הרשאות: קריאה (r), כתיבה (w), הרצה (x).
- **חלונות:** שימוש ב-ACL מלא וגמיש, תמיכה בהורשה (Inheritance) של הרשאות מתיקיית האב.
- **אנדרואיד (יכולות):** כל אפליקציה רצה כמשתמש נפרד. גישה למשאבים (מצלמה, אנשי קשר, רשת) מוגנת על ידי "יכולות" (Permissions) שהמשתמש מאשר בעת ההתקנה.

מנגנון SELinux: תוספת ללינוקס הממששת **בקרת גישה מנדטורית (MAC - Mandatory Access Control)**.
- לכל משאב ותהליך יש "תווית אבטחה" (Label).
- מודול בגרעין בודק כל גישה מול מדיניות מרכזית קשוחה, שלא ניתנת לשינוי על ידי המשתמשים (בניגוד ל-DAC - Discretionary Access Control הרגיל).

מנגנוני רישום (Logging): תיעוד פעולות לצרכי בקרה וזיהוי פריצות בדיעבד.
- רישום ניסיונות גישה (מוצלחים וכושלים), שינוי קבצים, התחברות ל-root.
- בחלונות יש מנגנון מובנה לרישום מפורט (Auditing). בלינוקס/יוניקס משתמשים בכלי עזר (כמו Tripwire) לזיהוי שינויים.

התחזות מותרת (Impersonation): מצבים בהם תהליך צריך לפעול עם הרשאות של משתמש אחר.
- שרת המבצע פעולה עבור לקוח צריך לרוץ עם הרשאות הלקוח.
- כלי ניהול (sudo) המאפשרים למשתמש רגיל להריץ פקודה כ-root.
- בחלונות: מנגנון Impersonation המאפשר לתהליך לאמץ זהות אחרת.

הגברה (Amplification): מצב שבו משתמש צריך לבצע פעולה הדורשת הרשאות שאין לו (למשל, שינוי סיסמה דורש כתיבה לקובץ הסיסמאות המוגן).
- **ביוניקס - SetUID:** תוכנית שהסיבית setuid דלוקה בה, רצה עם הרשאות ה**בעלים** שלה (למשל root) ולא עם הרשאות המשתמש שהריץ אותה. התוכנית אחראית לבצע רק את הפעולה המוגדרת והבטוחה.
- זהו מנגנון מסוכן: באג בתוכנית setuid יכול לאפשר לתוקף להשתלט על המערכת כולה.

וירטואליזציה ומיכלים (פרק 10.5)

מבוא: המניע לוירטואליזציה: בתיאוריה, מערכת ההפעלה אמורה לנהל את כל החומרה, להגן על תהליכים ולבודד אותם זה מזה (Protection, Isolation). מערכת הפעלה אחת אמורה להספיק להרצת כל היישומים. בפועל, המציאות שונה:
- **בידוד לא מושלם:** הפרדה בין תהליכים לא תמיד מספיקה (למשל, הרצת תוכנות של ארגונים מתחרים על אותו שרת).
- **תלויות סותרות:** תוכנות מסוימות מניחות שהן היחידות במחשב (שרתי HTTP, מסדי נתונים), או דורשות גרסאות שונות של ספריות מערכת.
- **תלות במערכת הפעלה:** יישומים מסוימים רצים רק על חלונות, אחרים רק על לינוקס, ולעיתים רק על גרסה ספציפית, גם אם החומרה זהה.
- **נדידה (Migration):** קשה להעביר תוכנית רצה ממחשב אחד לשני כדי לאזן עומסים או לחסוך אנרגיה. מערכות הפעלה רגילות לא תומכות בזה.

פתרון הוירטואליזציה: הוספת שכבת תוכנה הנקראת **רב-פקח** (Hypervisor או VMM - Virtual Machine Monitor) בין החומרה למערכות ההפעלה.
- מאפשר להריץ מספר מערכות הפעלה ("אורחות") על גבי חומרה אחת.
- כל מערכת הפעלה חושבת שהיא שולטת בחומרה באופן בלעדי.

שימושים נפוצים:
- **מחשוב ענן (IaaS):** משתמשים שוכרים מכונות וירטואליות לפי שעה, בתצורה המדויקת שהם צריכים.
- **תאימות לאחור/חוצת-פלטפורמות:** הרצת אופיס (חלונות) על לינוקס.
- **בדיקות תוכנה:** בדיקה על מגוון גרסאות מערכת הפעלה בקלות.
- **איחוד שרתים (Consolidation):** במקום להחזיק שרת פיזי נפרד לכל תפקיד (קבצים, דואר, הדפסה) שמנצל רק אחוז קטן מהחומרה, מאחדים אותם למכונה פיזית אחת שמריצה מספר מכונות וירטואליות.

טכניקות וירטואליזציה:

1. וירטואליזציה מלאה (גישת IBM, שנות ה-70): נבנה מעבד שמייצר חריג (Trap) בכל פעם שמנסים לבצע פקודה "מיוחסת" (פקודה שמשנה את מצב המכונה) במצב משתמש.
- מריצים את מערכת ההפעלה האורחת במצב משתמש (במקום במצב מיוחס).
- כל ניסיון שלה לגשת לחומרה גורם לחריג.
- הרב-פקח תופס את החריג, בודק אם הפעולה חוקית, ומבצע אותה עבור האורחת (סימולציה).
- הגישה דורשת תמיכה בחומרה (שכל הפקודות הרגישות ייצרו חריג). מעבדי x86 הישנים לא תמכו בכך (חלק מהפקודות נכשלו בשקט).

2. חיקוי (Emulation): תוכנה שמסמלצת את המעבד ואת כל ההתקנים (למשל Bochs, QEMU).
- כל פקודת מכונה מפוענחת בתוכנה ומבוצעת מול משתנים המייצגים אוגרים.
- **יתרון:** עובד על כל חומרה (אפשר להריץ חלונות על ARM).
- **חיסרון:** איטי מאוד.

3. שכתוב בינרי (VMWare - Binary Translation, 1998): פתרון לבעיית ה-x86 הישנה.
- הרב-פקח סורק את הקוד הבינרי של מערכת ההפעלה האורחת לפני הרצתו.
- הוא מחליף ("משכתב") פקודות בעייתיות בקריאות יזומות לרב-פקח.
- רוב הקוד רץ ישירות על המעבד (מהיר), ורק הפקודות המיוחסות מטופלות בתוכנה.
- לא דורש שינוי בקוד המקור של האורחת.

4. פארא-וירטואליזציה (Xen - Paravirtualization): ויתור על האשליה המושלמת של "חומרה אמיתית".
- משנים את קוד המקור של מערכת ההפעלה האורחת כך שתדע שהיא רצה במכונה וירטואלית.
- במקום לנסות לגשת לחומרה, היא מבצעת קריאות מפורשות לרב-פקח (Hypercalls).
- דורש גישה לקוד המקור (מתאים ללינוקס, בעייתי לחלונות סגורה).

5. תמיכת חומרה מודרנית (Intel VT-x / AMD-V): החל מ-2003, יצרני המעבדים הוסיפו מצב עבודה חדש למעבד (נוסף על Ring 0/3).
- דגל המציין אם המעבד מריץ "מארח" (Root mode) או "אורח" (Non-root mode).
- במצב אורח, פקודות מיוחסות גורמות לחריג אוטומטי שמעביר שליטה לרב-פקח.
- מאפשר וירטואליזציה מלאה ויעילה ללא שכתוב בינרי וללא שינוי קוד מקור (KVM, VirtualBox).

וירטואליזציה של זיכרון: ישנן שלוש רמות של כתובות:
1. וירטואלי-באורחת (של התהליך באורחת).
2. פיזי-באורחת (שהוא בעצם וירטואלי-במארחת).
3. פיזי אמיתי (של המחשב).

האתגר: כאשר האורחת משנה את טבלת הדפים שלה, זה לא משפיע על החומרה האמיתית.
- **פתרון תוכנה (טבלת צללים - Shadow Paging):** הרב-פקח מזהה כתיבות לטבלת הדפים של האורחת, ומעדכן במקביל "טבלת צללים" שממפה ישירות מוירטואלי-באורחת לפיזי-אמיתי. המעבד משתמש בטבלת הצללים.
- **פתרון חומרה (Nested Paging / EPT):** ה-MMU בחומרה יודע לבצע תרגום כפול דרך שתי הטבלאות (של האורח ושל המארח) באופן אוטומטי.

שימושים מתקדמים: היכולת לנהל מכונה כקובץ תוכנה מאפשרת:
- **הקפאה ושחזור (Snapshots):** שמירת מצב המכונה המדויק לדיסק וחזרה אליו (טוב לגיבוי ולניתוח כשלים).
- **נדידה (Migration):** העברת מכונה וירטואלית חיה בין שרתים פיזיים ללא השבתה.
- **מכשירים וירטואליים (Virtual Appliances):** הפצת תוכנה כ"מחשב שלם" מוכן לעבודה.

מיכלים (Containers): וירטואליזציה מספקת בידוד מצוין אך עם תקורה גבוהה (מריצים קרנל שלם לכל מכונה). **הפתרון:** מיכלים - בידוד ברמת מערכת ההפעלה (פתרון ביניים).

היסטוריה: chroot: פקודה ביוניקס (1979) שמשנה את תיקיית השורש (Root Directory) עבור תהליך מסוים. התהליך רואה רק תת-עץ של מערכת הקבצים ואינו יכול לצאת ממנו.

מרחבי שמות (Namespaces): בלינוקס מודרני, הרחיבו את רעיון ה-chroot לכל משאבי המערכת. ניתן ליצור מרחבי שמות נפרדים עבור:
- מספרי תהליכים (PID): תהליך במיכל יכול להיות PID 1.
- רשת: כרטיסי רשת, כתובות IP, טבלאות ניתוב ופורטים נפרדים.
- מערכת קבצים (Mount points).
- משתמשים (User IDs): משתמש יכול להיות root בתוך המיכל אך משתמש רגיל בחוץ.
- תקשורת בין תהליכים (IPC).
- שם מחשב (Hostname).

קבוצות בקרה (Cgroups): מנגנון להגבלת משאבים (כמות זיכרון, אחוז מעבד) לקבוצת תהליכים.

דוקר (Docker): מערכת פופולרית המשתמשת בטכנולוגיות הלינוקס הללו (Namespaces + Cgroups) כדי לארוז תוכנה יחד עם כל הספריות שלה ולהריץ אותה במיכל מבודד בקלות.

השוואה: וירטואליזציה מול מיכלים:
- **וירטואליזציה:**
  – ממשק: חומרה (פקודות מכונה).
  – תקורה: גבוהה (קרנל לכל מכונה).
  – גמישות: ניתן להריץ מערכות הפעלה שונות (ווינדוס על לינוקס).
  – אבטחה: בידוד חזק, משטח תקיפה קטן יחסית.
- **מיכלים:**
  – ממשק: קריאות מערכת (System Calls).
  – תקורה: נמוכה מאוד (כמעט כמו תהליך רגיל).
  – גמישות: חייבים להשתמש באותו קרנל (לינוקס על לינוקס).
  – אבטחה: בידוד חלש יותר, משטח תקיפה רחב (כל הקרנל המשותף חשוף).

טענות ממבחנים

כללי
- **Overhead:** זמן CPU או זיכרון שהמערכת "מבזבזת" על ניהול עצמי במקום הרצת תוכניות.
- בקורס הוגדר שהמעבד קורא מטבלת הדפים (Page Table), למרות שחומרתית זה משתנה.
- בחזרה מ-PAGE_FAULT, התהליך יבצע שוב את **אותה הפקודה** שגרמה לחריגה.
- **ELOOP:** שגיאה בפענוח נתיב (Pathname) עקב יותר מדי Symlinks (לרוב מעגל אינסופי: A→B→A). הקרנל עוצר למניעת קיפאון.
- הזיכרון עונה לכל בקשה על ה-Bus אם היא בתחום הכתובות הקיים.
- זמן ריצה (Wall Clock) של 10 שניות ≠ שימוש ב-CPU במשך 10 שניות.
- **הרשאות ב-Pipe:** בפקודה sudo echo abc □ foo, ה-sudo תקף רק לחלק שלפני ה-□. ההפניה מתבצעת ע"י ה-Shell (ללא root).
- שינוי הרשאות בטבלת הדפים מחייב TLB INVALIDATION.

3. ייצוגים בזיכרון (Memory Representations):

עקרונות היסוד לסידור משתנים בזיכרון (מה קבוע ומה תלוי ארכיטקטורה).
- **מצביעים ואובייקטים:** הכתובת &x מצביעה תמיד לכתובת של הבית **הראשון** (הכתובת הנמוכה ביותר) של האובייקט x. עובדה זו נכונה תמיד ואינה תלויה ב-Endianness או בכיוון המחסנית.
- **מערכים (Arrays):** מוקצים תמיד כבלוק זיכרון רציף ובסדר עולה.
  תמיד מתקיים &arr[i] < &arr[i+1].
  סדר האיברים בתוך המערך **אינו מושפע** מכיוון גדילת המחסנית או משיטת ה-Endianness. תא 0 תמיד יהיה בכתובת הנמוכה ביותר במערך.
- **סדר בתים (Endianness):** קובע את סדר הבתים **בתוך** אובייקט יחיד (למשל int).
  Little Endian: הבית הכי פחות משמעותי (LSB) יושב בכתובת הנמוכה ביותר.
  Big Endian: הבית הכי משמעותי (MSB) יושב בכתובת הנמוכה ביותר.
- **כיוון המחסנית (Stack Direction):** משפיע רק על המיקום היחסי של משתנים **נפרדים** שהוגדרו בזה אחר זה (למשל, האם int a יישב בכתובת גבוהה או נמוכה מ-int b).
  אם המחסנית גדלה לכתובות נמוכות (נפוץ): משתנה שהוגדר מאוחר יותר יישב בכתובת נמוכה יותר.

תהליכים וחוטים (Processes & Threads)
- **PPID:** תהליך יתום מאומץ ע"י init, לכן ה-PPID שלו הוא 1.
- **Exception ב-User Mode:** החומרה חייבת לשמור את ה-Program Counter (PC) (ולרוב SP) לפני הקריאה ל-Handler, כי המידע יאבד ברגע שה-Handler ירוץ.
- **Signal Handler:** משותף לכל החוטים בתהליך. שינוי בחוט אחד משפיע על כולם.
- **החלפות הקשר:** ייתכן Context Switch ללא Mode Switch ולהפך.
- **thread_cond_wait:** חייבת להיות פעולה אטומית למניעת DEADLOCK.
- **File Descriptor (FD):** משאב ברמת התהליך. סגירה בחוט אחד סוגרת לכולם.
- **אטומיות:** X++ אטומי (פקודת חומרה יחידה), בעוד X=X+1 אינו אטומי (קריאה-שינוי-כתיבה) וחשוף ל-Race Condition.
- **mutex שאיננו רקורסיבי בקריאת לנעול כאשר המנעול נעולה גורמת לdeadlock**

קריאות מערכת (System Calls)
- **dup2:** משכפלת FD. מחברת מספר FD ספציפי ל"צינור" (Pipe/Socket) קיים.
- **Pipe:** קריאה מצינור ריק → חוסמת (Blocking). כתיבה לצינור מלא → חוסמת (Blocking).
- **get_user(x, ptr):** מעתיקה מ-User ל-Kernel. בודקת תקינות (access_ok).
- **put_user(x, ptr):** מעתיקה מ-Kernel ל-User. בודקת הרשאות כתיבה.

זיכרון ווירטואלי (Virtual Memory)
- מרחב הכתובות הווירטואלי רציף (לא ניתן למפות 30MB בחור של 10MB).
- **חזרה מ-Page Fault:** התהליך יבצע שוב את הפקודה שגרמה לשגיאה. תרחישים: Anonymous (הקצאה+איפוס), Backed-file (מיפוי ל-Page Cache קיים), או Swap (הבאה מדיסק).
- **הליכה בטבלה היררכית:** כל רמה = גישה לזיכרון. 64 ביט = 4 רמות. כישלון ברמה כלשהי עוצר את התהליך מיידית.
- **מיפויים אפשריים:** אותו דף פיזי יכול להיות: (א) ממופה לקריאה וכתיבה במקומות שונים; (ב) משותף בין תהליכים; (ג) כתובות וירטואליות עוקבות ממופות לפיזיות לא עוקבות.
- **Offset:** זהה בווירטואלי ובפיזי (לא נשמר בטבלה).
- **אובייקט FILE בקרנל:** נחוץ לשמירת Offset ולתמיכה ב-Socket/Pipe (לא רק Inode).
- **Valid Bit:** 1=נמצא ב-RAM, 0=Page Fault. **Dirty Bit:** 1=שונה מאז הטעינה (חייב שמירה לדיסק), 0=זהה לדיסק.
- **mmap:** ממפה וירטואלי לקובץ. אם הקובץ כבר ב-RAM (ע"י open/read אחר), המיפוי יצביע ל-Page Cache הקיים ללא שכפול פיזי.

מערכת קבצים, התקנים ו-RAID
- **FFS:** מחלק ל-Cylinder Groups. שומר Data+Meta קרוב. קובץ ותיקיית האב קרובים. תתי-תיקיות מפוזרות לפיזור עומס.
- **מחיקת Inode:** מתבצעת סופית רק כש: (1) Hard Link Count הוא 0 **וגם** (2) אין תהליכים המחזיקים FD פתוח לקובץ.
- לא ניתן לבצע Hard Link לספרייה.
- **תרגום כתובות בין מערכת הקבצים לדיסק הפיזי**
  – **מערכת הקבצים:** רואה את הבלוקים כמספרים סידוריים המתחילים ב-0 בתוך המחיצה (Partition) שלה בלבד.
  – **הדיסק (SATA Bus):** מקבל כתובת אבסולוטית הנקראת Block (Logical LBA Address), המתייחסת למיקום הממשי מתחילת הכונן הפיזי כולו.
- **Polling:** עדיף על Interrupts רק כשההתקן מהיר מאוד.
- **RAID:** ריבוי דיסקים במערך אחד מגדיל את הסיכוי לכשל (אובדן נתונים), אלא אם יש יתירות.
- **סיגנלים:** Handler יכול לטפל איך שרוצה. SIGTERM = בקשה מנומסת לסיום (מאפשר ניקוי).
- אפשר לתקשר בין תהליכים באמצעות הסיגנלים SISUR1 SIGSUR2.

תקשורת (Networking)
- **Packet-loss:** נגרם מעומס (Congestion) בנתבים או תקלה פיזית.
- **כתובות פרטיות (Private IP):** A: 10.0.0.0, B: 172.16-31, C: 192.168.0.0.
- **Coexistence:** ניתן להריץ שרת TCP ושרת UDP על אותו פורט (למשל 80) במקביל.
- **זיהוי Socket:** מזוהה ע"י 5 פרמטרים: {Proto, SrcIP, SrcPort, DstIP, DstPort}. שרת יחיד בפורט 80 יכול לנהל אלפי חיבורים שונים.
- **send:** חוסם רק עד העתקה ל-Kernel Buffer. חזרה ≠ הגעה ליעד.
- **close:** סגירת FD האחרון שולחת FIN.
- **יציאה מרשת ביתית (NAT):** פקטה יוצאת עם DstIP של היעד המרוחק, אך DstMAC של הנתב (Router).
- **TCP Reliability:** אם הודעה 1 לא התקבלה, הודעה 2 שנשלחה אחריה באותו חיבור לא תועבר לאפליקציה (תיתקע ב-Buffer) עד שהראשונה תגיע (Head-of-Line Blocking).
- **הקמת שרת:** bind אינו חובה (מערכת ההפעלה תבחר פורט). לקוח יכול להתחבר (connect) מיד אחרי שהשרת עשה listen, גם לפני accept.

מידע מתרגולים

קבצים (Recitation 1):
- void exit(int status): מסיים את התהליך הקורא מיידית. סטטוס 0 הוא הצלחה, אחרת כישלון (לרוב 1). סוגר את כל ה-fd ומרוקן חוצצים.
- ssize_t read(int fd, void* buf, size_t count): מנסה לקרוא עד count בתים לתוך buf.
  – מצליח ← מחזיר מספר בתים שנקראו. נכשל ← מחזיר −1 ומעדכן errno.
- ssize_t write(int fd, const void* buf, size_t count): מצליח ← מספר בתים שנכתבו (ייתכן שפחות מ-count). נכשל ← -1.

מודולי ליבה (Recitation 3): מודולי ליבה נטענים (LKM): קוד שניתן לטעון דינמית לקרנל (כמו דרייברים). תכנות בקרנל מורכב ורגיש, וגורם לקריסות (Panic).
- **פונקציית אתחול:** static int __init hello_init(void).
  – static: הפונקציה מקומית לקובץ.
  – __init: מאקרו לטיפול מיוחד, הזיכרון משוחרר לאחר הטעינה.
  – חזרה: 0 להצלחה, אחרת המודול לא ייטען.
- **פונקציית ניקוי:** static void __exit hello_cleanup(void). המאקרו __exit מסמן קוד ניקוי.
- **פקודות:**
  – sudo insmod hello.ko: טעינת המודול (קורא ל-init).
  – sudo rmmod hello: הסרת המודול לפי שם (קורא ל-cleanup).
  – lsmod: הצגת מודולים טעונים.
- **Panic:** עצירת חירום. נמנעים ע"י: אי-אמון בקלט, בדיקת NULL, וטיפול בשגיאות. במקרה של Panic - אתחול מחדש.

מנהלי התקן (Recitation 4):
- int register_chrdev(uint major, char* name, file_operations* fops):
  – major: מספר רצוי (0 להקצאה דינמית). fops: מבנה המקשר פעולות לפונקציות.
  – מחזיר 0 בהצלחה, -1 בכישלון.
- void unregister_chrdev(...): ביטול הרישום.
- mknod <name> <type> <major> <minor>: יצירת התקן חדש. סוג: "b"/"c".
- ioctl(file*, cmd, arg): לפקודות מיוחדות שלא מכוסות ע"י קריאה/כתיבה. החזרה תלויה במימוש (<0 לשגיאה).
- **struct file_operations:** ממפה פעולות (open, close, read, write, ioctl...) לפונקציות בדרייבר.

תהליכים (Recitation 5):
- int execvp(path, argv[]): מחליף את "תמונת" התהליך בחדשה.
  – argv[0] הוא שם הקובץ, האיבר האחרון חייב להיות NULL.
  – מחליף זיכרון, מחסנית, ערימה. לא חוזר אם הצליח.
- wait(status) / waitpid(pid, status, options): האב ממתין לשינוי מצב הבן. מחזיר -1 אם אין בנים או כולם טופלו.
- **המתנה לכל הבנים:** while(-1 != wait(&status));. חובה לאסוף בנים כדי למנוע זומבים. אם האב מת קודם, הבן מאומץ ע"י תהליך 1.
- int kill(pid, sig): שליחת סיגנל לתהליך. 0 בהצלחה.

צינורות ומיפוי זיכרון (Recitation 6): צינורות (Pipes):
- int pipe(int pipefd[2]): 0 להצלחה. fd[0] לקריאה, fd[1] לכתיבה.
- **כתיבה ללא קורא:** נשלח סיגנל SIGPIPE והתהליך מסתיים (ברירת מחדל).
- **קריאה ללא כותב:** לא קורה כלום (לא נתקע), הקריאה חוזרת עם 0 לציון EOF.

מיפוי זיכרון (mmap):
- mmap(start, length, prot, flags, fd, offset):
  – start: כתובת מועדפת (או NULL). prot: הרשאות (rwx).
  – flags: MAP_SHARED (שינויים נראים לאחרים), MAP_PRIVATE (עותק פרטי COW), MAP_ANONYMOUS (ללא קובץ, מאופס).
- munmap(start, len): מוחק מיפוי, שוטף שינויים לדיסק.
- msync(start, len, flags): סנכרון לדיסק. דגלים: MS_SYNC (מיד), MS_ASYNC (מתוזמן).

חוטים (Recitation 7):
- int thrd_create(thrd_t* t, func, arg): יצירת חוט. void* מצביע להכל.
- void thrd_exit(res): מסיים את החוט הנוכחי בלבד (לא משפיע על אחרים). שים לב: exit() הורג את כל התהליך!
- thrd_current(): מחזיר ID. thrd_equal(t1, t2): בודק שוויון (לא אפס אם שווים).
- int thrd_join(th, res*): ממתין לסיום חוט. כל חוט יכול לחכות לכל חוט.
- ערכי חזרה (thrd_..): success, nomem, timedout, busy, error.

מנעולים (Recitation 8):
- _Atomic(int): משתנים אטומיים (מימוש ידידותי ל-compare & swap).
- mtx_init(mutex, type): סוגים: mtx_..$plain, timed(nonrecursivesupporttimeout), recursive$ (ושילובים).
- mtx_destroy: התנהגות לא מוגדרת אם יש חוטים שמחכים!
- **נעילה:** mtx_lock (חוסם), mtx_trylock (מנסה פעם אחת, נכשל אם תפוס), mtx_unlock.

משתני תנאי (Condition Variables):
  – cnd_init / cnd_destroy.
  – cnd_wait(cond, mutex): חייב להחזיק במנעול. משחרר אטומית והולך לישון. כשמתעורר נועל שוב. יכול להתעורר סתם (spurious).
  – cnd_signal: מעיר חוט אחד (שרירותי). אין צורך במנעול.
  – cnd_broadcast: מעיר את כולם.

מערכת הקבצים (Recitation 9):
- chmod <groups><op><perm> file: למשל u+x.
- קבוצות: u = user, g = group, o = others, a = all □. פעולות: +, -, =. הרשאות: r(4), w(2), x(1).
- **קישור קשיח:** ln file link. **קישור סימבולי:** ln -s file link.

רשתות (Recitation 10):
- **המרת סדר בתים (Endian):** htonl, htons (לרשת), ntohl, ntohs (למארח).
- socket(family, type, proto): יוצר נקודת קצה.
  – AF_INET (v4), AF_INET6 (v6), AF_UNIX.
  – SOCK_STREAM (TCP), SOCK_DGRAM (UDP).
  – מחזיר descriptor.
- bind(sockfd, addr, len): קישור לכתובת ופורט. מחזיר 0 בהצלחה -1 בכישלון.
- listen(sockfd, backlog): סימון כמאזין לקבלת חיבורים. מחזיר 0 בהצלחה -1 בכשלון
- accept(sockfd, peer_addr, len): המתנה לחיבור. יוצר סוקט **חדש** לשיחה. מחזיר את הסוקט החדש בהצלחה ובכשלון -1
- connect(sockfd, addr, len): התחברות לשרת. מחזיר 0 בהצלחה -1 בכשלון.
- **מבני נתונים:** sockaddr_in (משפחה, פורט, in_addr), in_addr (32 ביט IP). ל-IPv6 מבנים מקבילים.
- read/write: ייתכן שיקראו/יכתבו פחות מהמבוקש (בלוק חלקי).
- send/recv: כמו הנ"ל עם דגלים.
- close: סגירה. קריאה מסוקט סגור בצד השני מחזירה 0 (EOF).

אבטחה ודיבאג (Recitations 11-12): אבטחה:
- **Sticky bit (1):** chmod 1777. מונע מחיקת קבצים של אחרים בתיקיות ציבוריות (/tmp).
- **SetUID (4):** chmod 4777. הרצה עם הרשאות הבעלים (למשל root). דוגמה: passwd.
- **SetGID (2):** chmod 2777. הרצה עם הרשאות הקבוצה.

ניפוי שגיאות (Debugging):
- ptrace(op, pid, addr, data): מאפשר שליטה בתהליך אחר.
- PTRACE_TRACEME: הבן מבקש מהאב לדבג אותו (לפני exec).
- אחרי exec, התהליך נעצר (SIGTRAP) והאב מקבל הודעה.

מילון סיגנלים ודגלים (Signals & Flags)

מקרא: Term=סיום, Core=סיום+Dump, Stop=השהייה, Ign=התעלמות.

1. רשימת סיגנלים (Signals List):
- **בלתי ניתנים לתפיסה/התעלמות (Kill/Stop):**
  SIGKILL (9): הריגה מיידית וסופית. מנקה משאבים בכוח.
  SIGSTOP (19): הקפאת ריצה מיידית (השהייה).
- **מקלדת (Keyboard):**
  SIGINT (2): Ctrl+C. פסיקה רגילה (Term).
  SIGQUIT (3): Ctrl+\. יציאה עם Core Dump.
  SIGTSTP (20): Ctrl+Z. בקשת השהייה (ניתן לתפיסה).
  הערה: **Ctrl+D** אינו סיגנל! זהו EOF.
- **ניהול תהליכים ושגיאות:**
  SIGTERM (15): בקשת סיום "מנומסת". ברירת המחדל של kill.
  SIGCHLD: נשלח לאב כשבן סיים. (דיפולט: Ign, טריגר ל-wait).
  SIGSEGV: גישה לזיכרון לא חוקי (Core).
  SIGPIPE: כתיבה לצינור שאין לו קורא (Term).

2. דגלים ומקרואים (Flags & Macros):
- **התנהגות (Disposition):**
  SIG_DFL: החזרת הטיפול לברירת המחדל (Default).
  SIG_IGN: התעלמות מהסיגנל (Ignore).
- **דגלי sigaction (sa_flags):**
  SA_SIGINFO: מאפשר קבלת מידע נוסף בהנדלר (כמו PID השולח).
  SA_RESTART: הפעלה מחדש אוטומטית של Syscalls (מונע EINTR).
- **בדיקת סטטוס יציאה (אחרי wait):**
  WIFEXITED(s): האם הסתיים תקין?
  WEXITSTATUS(s): שליפת קוד היציאה (ה-int).
  WIFSIGNALED(s): האם נהרג ע"י סיגנל?

3. שגיאות נפוצות (Errno):
- EINTR: קריאת מערכת נקטעה ע"י סיגנל (כי לא שמנו SA_RESTART).
- ECHILD: ניסיון לעשות wait כשאין בנים (או כולם נאספו).